

UNIVERSITY OF OSLO

Faculty of mathematics and natural sciences

Exam in: STK-IN4300/9300 — Statistical Learning
Methods in Data Science

Day of examination: Tuesday, 28 November 2023

Examination hours: 15.00–19.00

This problem set consists of 5 pages.

Appendices: None.

Permitted aids: None.

Please make sure that your copy of the problem set is complete before you attempt to answer anything.

Problem 1 Fitting a Regression Function

Consider a regression setting and assume an additive error model:

$$Y = f_{\theta}(X) + \epsilon,$$

where θ denotes the model parameters defining the regression function f_{θ} and $\epsilon \sim N(0, \sigma^2)$ is the error term.

- (a) For the purpose of estimating θ from data, describe (i) the least squares method and (ii) the maximum likelihood method.
- (b) Show that the least squares method and the maximum likelihood method are equivalent under the considered assumptions described above. Hint: recall that the probability density function of a random variable $Z \sim N(\mu, \sigma^2)$ is given by:

$$f(z) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2}\left(\frac{z-\mu}{\sigma}\right)^2}, \quad z \in \mathbb{R} \text{ and } \mu \in \mathbb{R}, \sigma > 0.$$

Problem 2 Bias-Variance Tradeoff

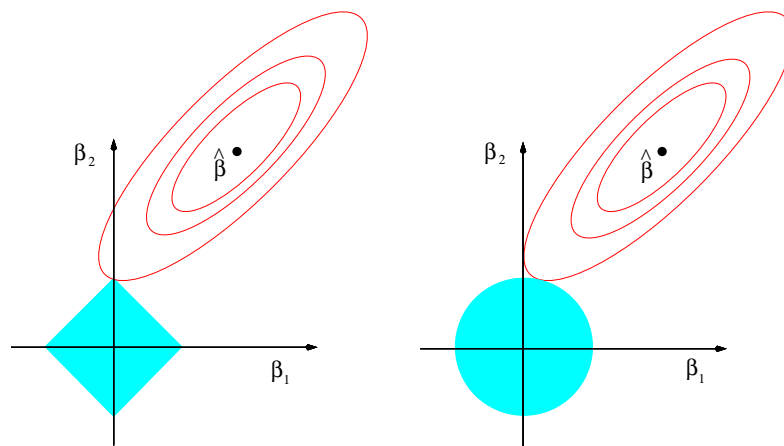
Consider a regression setting with an additive error model $Y = f(X) + \epsilon$, where $\mathbb{E}[\epsilon] = 0$ and $\text{Var}[\epsilon] = \sigma_{\epsilon}^2$.

- (a) Using the squared-error loss, show that the expected prediction error of a regression fit $\hat{f}(X)$ at input point $X = x_0$ can be decomposed as:

$$\mathbb{E}[(Y - \hat{f}(x_0))^2 | x_0] = \sigma_{\epsilon}^2 + (\mathbb{E}[\hat{f}(x_0)] - f(x_0))^2 + \mathbb{E}[(\hat{f}(x_0) - \mathbb{E}[\hat{f}(x_0)])^2],$$

and describe what the three terms on the right-hand side of the above equation represent. Hint: recall that the variance of a random variable Z with $\mathbb{E}[Z] = \mu$ is defined as $\text{Var}[Z] = \mathbb{E}[(Z - \mu)^2]$.

(Continued on page 2.)

Figure 1: From Hastie et al. (2009). *The Elements of Statistical Learning*.

- (b) Assuming that $\hat{f}(X)$ is a k -nearest-neighbor regression fit, the above expected prediction error expression takes the specific form of

$$\mathbb{E}[(Y - \hat{f}(x_0))^2 | x_0] = \sigma_\epsilon^2 + \left(f(x_0) - \frac{1}{k} \sum_{\ell=1}^k f(x_{(\ell)}) \right)^2 + \frac{\sigma_\epsilon^2}{k},$$

where $x_{(\ell)}$ denotes the ℓ :th nearest neighbor of x_0 in the training data. Describe the role of k in controlling the complexity of the model and the associated bias-variance tradeoff.

Problem 3 Linear Regression with Shrinkage

Assume a linear regression model $Y = \beta_0 + \sum_{j=1}^p \beta_j X_j + \epsilon$ with p input variables. Given some training data, we can define a shrinkage estimator of the model parameters as the solution to the following optimization problem:

$$\underset{\beta}{\operatorname{argmin}} \left\{ \sum_{i=1}^N (y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j)^2 \right\},$$

subject to $R(\beta) \leq t$,

where $R(\beta) = \sum_{j=1}^p \beta_j^2$ for ridge regression and $R(\beta) = \sum_{j=1}^p |\beta_j|$ for lasso.

- (a) Describe briefly the general idea behind shrinkage methods, such as ridge regression and lasso, and also how t is connected to the bias-variance tradeoff for the fitted models.
- (b) Consider Figure 1 which shows the contours (red ellipses) of the objective function around the least-squares solution $\hat{\beta}$ and the constraint regions of lasso and ridge regression (solid blue areas) for the above optimization problem in the case of two input variables. Based on Figure 1:

(Continued on page 3.)

- (i) Explain the key difference between ridge regression and lasso in terms of the model they produce.
- (ii) Explain how changing t would affect the constraint regions and the resulting parameter estimates.

Problem 4 Model Selection and Evaluation

- (a) Consider the following strategy for performing a regression analysis in a case where there is a very large number of input variables $X_1, \dots, X_p$:
1. Initial screening: Find a good subset of predictors by including the $q \ll p$ input variables that are most strongly correlated with the response variable.
 2. K -fold cross-validation: For $k = 1, \dots, K$, train a model for predicting the response given the q input variables (from Step 1) using all data except fold k and compute a fold-specific estimate of the test error based on the observations in fold k .
 3. Average the fold-specific test errors obtained in Step 2 to obtain a cross-validation estimate of the test error.

Is the averaged test error obtained from the above procedure a good estimate of the expected prediction error of a new test observation from the same distribution? If not, what would you do differently to improve it?

- (b) Let $\mathcal{T}$ denote a set of training data containing N joint observations of the input-output pair (X, Y) and let (X^0, Y^0) denote a new test data pair (all generated from the same $p(X, Y)$). Further let $\hat{f}(X)$ be a model fitted on the training data. Given some loss function $L(y, f(x))$, consider the following definitions of errors for the fitted model:

$$(i) \text{Err}_{\mathcal{T}} = \mathbb{E}_{X^0, Y^0}[L(Y^0, \hat{f}(X^0)) | \mathcal{T}]$$

$$(ii) \text{Err} = \mathbb{E}_{\mathcal{T}}[\mathbb{E}_{X^0, Y^0}[L(Y^0, \hat{f}(X^0)) | \mathcal{T}]]$$

$$(iii) \text{Err}_{\text{in}} = \frac{1}{N} \sum_{i=1}^N \mathbb{E}_{Y^0}[L(Y_i^0, \hat{f}(x_i)) | \mathcal{T}]$$

Explain what errors (i)–(iii) measure and what they are used for in the context of model selection and evaluation.

Problem 5 Bagged Trees and Random Forest

A regression and classification tree model can formally be expressed as

$$T(x; \Theta) = \sum_{j=1}^J \gamma_j I(x \in R_j),$$

(Continued on page 4.)

where $\Theta = \{\gamma_j, R_j\}_{j=1}^J$ contains all the model parameters describing the tree, or its regions R_j , as well as the associated region-specific constants γ_j .

- (a) By using bootstrap aggregation, or bagging, we can construct a so-called bagged tree estimate:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B T(x; \Theta_b).$$

How is the above estimate constructed and why does it in general improve the accuracy of a single-tree model?

- (b) Let $Z_1, \dots, Z_B$ be B identically distributed random variables with variance σ^2 . If the variables are dependent, with a positive correlation ρ , one can show that:

$$\text{Var}\left[\frac{1}{B} \sum_{b=1}^B Z_b\right] = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$

Explain why the bagged tree model is limited by the above result and how random forest can be considered an improved version of the bagged tree model.

- (c) By letting $B \rightarrow \infty$, we obtain the limiting form of the random forest regression estimator:

$$\hat{f}_{\text{rf}}(x) = \mathbb{E}_{\Theta | \mathbf{Z}}[T(x; \Theta)],$$

where $\mathbf{Z}$ denotes the training data. Further, it can be shown that the total variance of a tree can be decomposed into a sum of two terms:

$$\text{Var}_{\Theta, \mathbf{Z}}[T(x; \Theta)] = \text{Var}_{\mathbf{Z}}\left[\mathbb{E}_{\Theta | \mathbf{Z}}[T(x; \Theta)]\right] + \mathbb{E}_{\mathbf{Z}}\left[\text{Var}_{\Theta | \mathbf{Z}}[T(x; \Theta)]\right].$$

What does the total variance and the two terms on the right-hand side of the above equation represent?

Problem 6 Boosting

Consider a two-class classification problem where the binary output variable is coded as $Y \in \{-1, 1\}$.

- (a) For the considered problem, it can be shown that the AdaBoost algorithm in Figure 2 is equivalent to forward stagewise additive modelling under the exponential loss function:

$$L(y, f(x)) = e^{-yf(x)},$$

and, as a consequence of this, one can show that AdaBoost is seeking to estimate the population minimizer:

$$f^*(x) = \underset{f(x)}{\text{argmin}} \left\{ \mathbb{E}_{Y|x} [e^{-Yf(x)}] \right\}.$$

Derive an expression for $f^*(x)$ and explain, based on the expression, why the classification rule of AdaBoost is reasonable.

(Continued on page 5.)

Algorithm 10.1 *AdaBoost.M1.*

1. Initialize the observation weights $w_i = 1/N$, $i = 1, 2, \dots, N$.
2. For $m = 1$ to M :
 - (a) Fit a classifier $G_m(x)$ to the training data using weights w_i .
 - (b) Compute

$$\text{err}_m = \frac{\sum_{i=1}^N w_i I(y_i \neq G_m(x_i))}{\sum_{i=1}^N w_i}.$$
 - (c) Compute $\alpha_m = \log((1 - \text{err}_m)/\text{err}_m)$.
 - (d) Set $w_i \leftarrow w_i \cdot \exp[\alpha_m \cdot I(y_i \neq G_m(x_i))]$, $i = 1, 2, \dots, N$.
3. Output $G(x) = \text{sign} \left[\sum_{m=1}^M \alpha_m G_m(x) \right]$.

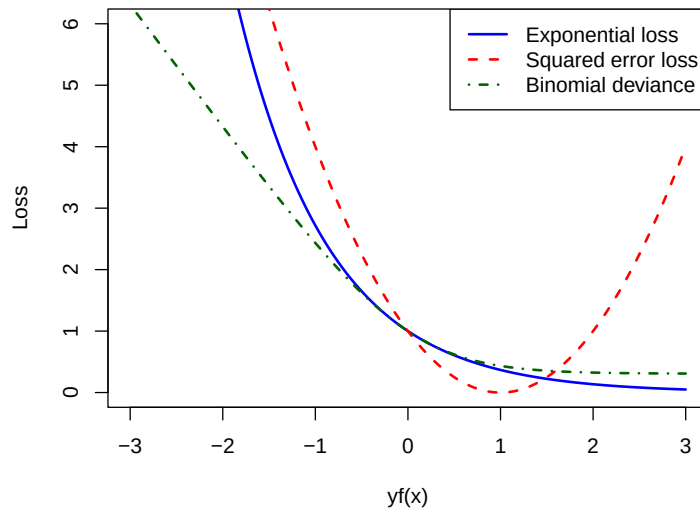
Figure 2: From Hastie et al. (2009). *The Elements of Statistical Learning*.

Figure 3: Different loss functions plotted against the margin.

- (b) When classifying a $-1/1$ response using the classification rule $\text{sign}[f(x)]$, the margin $yf(x)$ plays a role analogous to the residuals in regression. Consider the exponential loss, squared error loss and binomial deviance, which are plotted in Figure 3 against the margin. Discuss the strengths and weaknesses of the considered loss functions in comparison to each other.

THE END - GOOD LUCK!

UNIVERSITY OF OSLO

Faculty of mathematics and natural sciences

Exam in: STK-IN4300/STK-IN9300 — Statistical learning methods in Data Science

Day of examination: Monday, December 5th, 2022

Examination hours: 09:00 – 13:00

This problem set consists of 5 pages.

Appendices: None.

Permitted aids: None.

Please make sure that your copy of the problem set is complete before you attempt to answer anything.

Problem 1 Bias - variance trade-off (20 pt.)

1a L_2 -Boosting (10 pt.)

Consider the following results from Proposition 3 and Theorem 1 in *Boosting with the L_2 loss* (Bühlmann & Yu, 2003):

(i) $\text{bias}^2(m, \mathcal{S}; f) = \frac{1}{n} f^T U \text{diag}((1 - \lambda_k)^{2m+2}) U^T f$

(ii) $\text{var}(m, \mathcal{S}; \sigma^2) = \frac{\sigma^2}{n} \sum_{k=1}^n (1 - (1 - \lambda_k)^{m+1})^2$

(iii) if $\frac{\mu_k^2}{\sigma^2} > \frac{1}{(1 - \lambda_k)^2} - 1$ for all k with $\lambda_k < 1$, then $\text{MSE}_{\mathcal{B}_m} < \text{MSE}_{\mathcal{S}}$

where m is the number of boosting iterations; λ_k are the eigenvalues of the symmetric smoothing operator $\mathcal{S}$, such that $0 < \lambda_k \leq 1$ for $k = 1, \dots, n$; the boosting operator $\mathcal{B}_m$ is such that $\mathcal{B}_m = U D_m U^T$ with $D_m = \text{diag}(1 - (1 - \lambda_k)^{m+1})$ and $U U^T = U^T U = I$; $\mu = U^T f$ (μ represents f in the coordinate system of the eigenvectors of $\mathcal{S}$).

Clarify the implications of points (i), (ii) and (iii). Do these results hold in general?

1b k-Nearest Neighbours (5 pt.)

Describe how the choice of the number of neighbours k in a kNN procedure is connected to the bias-variance trade-off.

1c Smoothing splines (5 pt.)

Considering smoothing splines, explain the concept of *trimming down the hills and filling in the valleys* and relate it to the bias-variance trade-off.

(Continued on page 2.)

Problem 2 Ensemble methods (40 pt.)

2a Bagging (10 pt.)

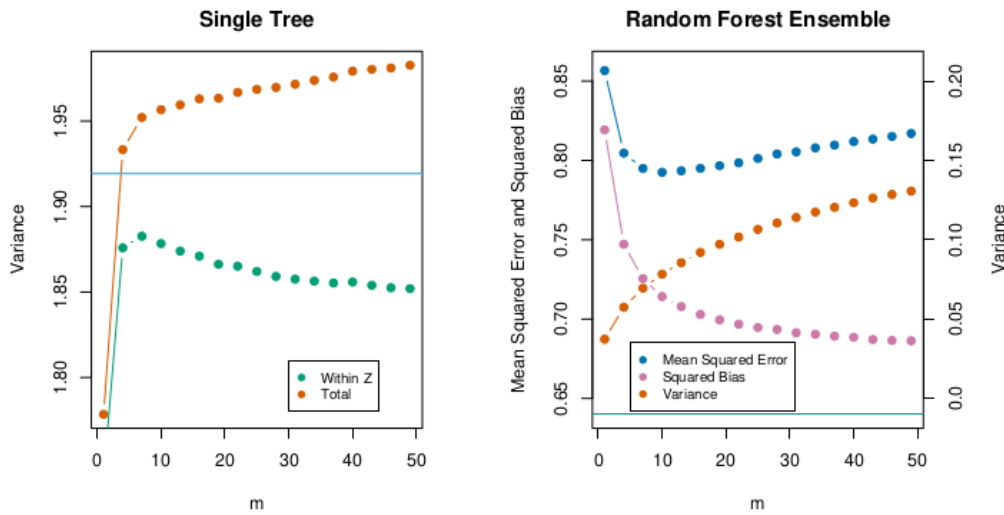
Describe the general procedure for tree-based bagging for a classification problem, both with consensus and probability. Highlight the strengths and limitations of the method, in terms of expected prediction error and tree correlation.

2b Random forest

Consider the total variance $\text{Var}_{\Theta, Z}[T(x; \Theta(Z))]$ of a single tree in a Random Forest, which can be decomposed as

$$\text{Var}_{\Theta, Z}[T(x; \Theta(Z))] = \text{Var}_Z[E_{\Theta|Z}[T(x; \Theta(Z))]] + E_Z[\text{Var}_{\Theta|Z}[T(x; \Theta(Z))]].$$

- Specify each of the two terms on the right-hand side, and discuss their behaviour in relation to the tuning parameter m , which is the size of the random input subgroup considered at each split; **(5 pt.)**
- Comment the following figure (Hastie, Tibshirani & Friedman, 2009, The Elements of Statistical Learning, Figure 15.10), linking it to your previous discussion. **(5 pt.)**



Left horizontal line: average variance of a single fully grown tree (without bootstrap sampling).

Right horizontal line: average squared-bias of a fully grown tree.

2c Boosting

Show that AdaBoost is a forward stagewise additive modelling procedure

$$\hat{f}^{[m]}(x_i) = \sum_{k=1}^m \hat{\beta}^{[k]} \hat{g}^{[k]}(x_i) = \hat{f}^{[m-1]} + \hat{\beta}^{[m]} \hat{g}^{[m]}(x_i)$$

(Continued on page 3.)

with exponential loss function $L(y, f(x)) = \exp[-y f(x)]$, considering a two-class classification problem with $y_i \in \{-1, 1\}$ for $i = 1, \dots, n$. In particular:

- Show that the classifier $\hat{g}$ at each step is the minimizer of the weighted misclassification error; **(8 pt.)**
- Derive the expression for the voting weights: **(8 pt.)**

$$\beta^{[m]} = \frac{1}{2} \log \frac{1 - \text{err}^{[m]}}{\text{err}^{[m]}};$$

Hint: at step m , $\hat{f}^{[m-1]}$ does not depend on the minimizing parameters. Note that

$$y_i g(x_i) = \begin{cases} 1 & \text{if } y_i = g(x_i) \\ -1 & \text{if } y_i \neq g(x_i) \end{cases}$$

- Boosting, therefore, can be seen as an iterative procedure to minimize a loss function. Explain why one should not treat this problem as a standard numerical minimization given the training data. What is the crucial point in the procedure of minimizing the loss with boosting? **(4 pt.)**

Problem 3 Projection Pursuit and Neural Networks (20 pt.)

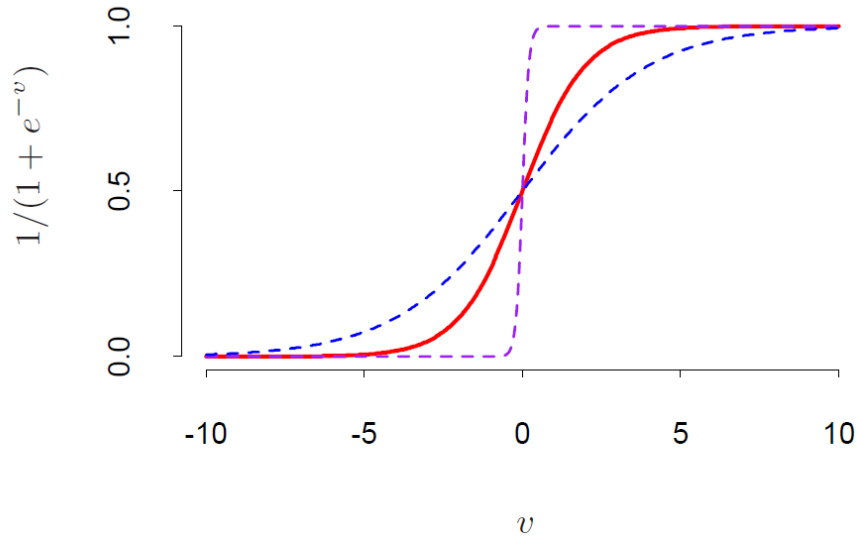
3a Projection pursuit (10 pt.)

Briefly describe the concept of projection pursuit. Connect it to the concept of a neural network and create a formal connection between the output of a neural network and the output of a projection pursuit model.

3b Activation functions

- Explain the concept behind the activation function in a Neural Network **(5 pt.)**

(Continued on page 4.)



- The above figure (Hastie, Tibshirani & Friedman, 2009, The Elements of Statistical Learning, Figure 11.3) shows the sigmoid function $\sigma(sv) = 1/(1 + \exp(-sv))$ with $s = 1$ (red curve), where s controls the activation rate.
In the above figure, identify the curve for $s = 1/2$ and $s = 10$. Explain how s effects the activation function and which activation functions are obtained in the limit where s tends to 0 or ∞ . (5 pt.)

Problem 4 Penalized regression (20 pt.)

4a Smoothly Clipped Absolute Deviation (SCAD)

The coefficients for the SCAD are obtained as follows:

$$\hat{\beta}_{\text{SCAD}}(\lambda, \alpha) = \operatorname{argmin}_{\beta} \left\{ \frac{1}{2} \|y - \beta_0 \vec{1} - X\beta\|_2^2 + \lambda \sum_{j=1}^p p_j(\beta_j; \lambda, \alpha) \right\},$$

where

$$\frac{dp_j(\beta_j; \lambda, \alpha)}{d\beta_j} = \lambda \left\{ \mathbb{1}(|\beta_j| \leq \lambda) + \frac{(\alpha\lambda - |\beta_j|)_+}{(\alpha - 1)\lambda} \mathbb{1}(|\beta_j| > \lambda) \right\}$$

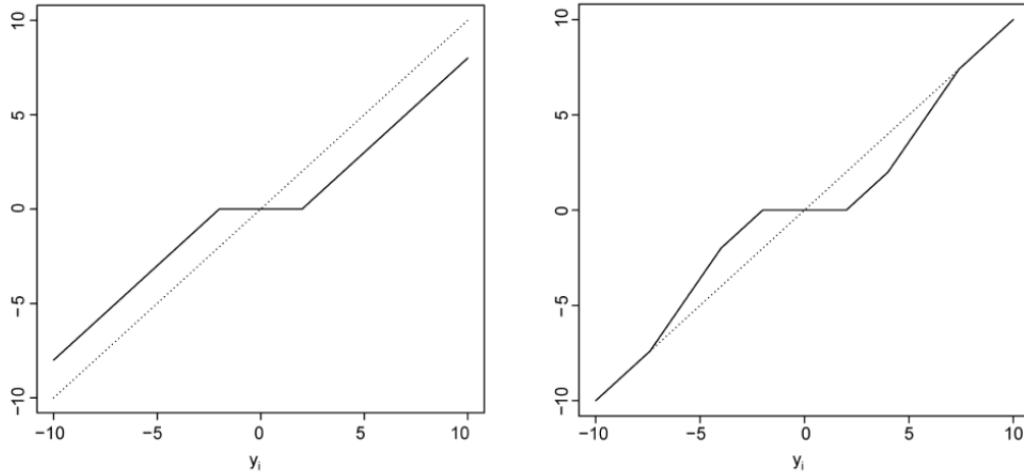
for $\alpha > 2$.

Briefly explain the motivation behind the SCAD Algorithm and its benefits over the standard lasso (5 pt.)

(Continued on page 5.)

4b Properties SCAD

Consider the following figure from *The adaptive Lasso and its oracle properties* (Zou, 2006), Figure 1, which shows a plot of thresholding functions for the lasso on the left and SCAD on the right. It compares the size of the coefficients for Ordinary Least Squares $\hat{\beta}_{OLS}$ against the size of coefficients in the case of the lasso or SCAD.



- Explain the figure above and what this tells you about the way lasso and SCAD shrink the regression coefficients β respectively **(5 pt.)**
- Describe the role and effect of α **(3 pt.)**
- What inequality does $|\beta|$ need to fulfill so that it becomes the β from the ordinary least squares estimate? **(2 pt.)**

4c Oracle properties (5pt.)

Briefly and qualitatively (using words and not necessarily mathematical notation), explain the two properties that an oracle estimator has.

UNIVERSITY OF OSLO

Faculty of mathematics and natural sciences

Exam in: STK-IN4300/STK-IN9300 — Statistical learning methods in Data Science

Day of examination: Friday, December 3rd, 2021

Examination hours: 9.00–13.00

This problem set consists of 6 pages.

Appendices: None.

Permitted aids: Any.

Please make sure that your copy of the problem set is complete before you attempt to answer anything.

Problem 1 Elastic Net

Consider the following formulation of Elastic Net,

$$\hat{\beta}_{\text{EL-Net}} = \underset{\beta}{\operatorname{argmin}} \left\{ \sum_{i=1}^N \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2 + \lambda_1 \sum_{j=1}^p |\beta_j| + \lambda_2 \sum_{j=1}^p \beta_j^2 \right\}$$

where $y_1, \dots, y_N$ is a continuous response; x_{ij} is the i -th observation ($i = 1, \dots, N$) of the j -th variable ($j = 1, \dots, p$); $\beta_0, \beta_1, \dots, \beta_p$ are regression coefficients; and λ_1, λ_2 are hyperparameters.

a (8 pt.)

Using this formulation, briefly describe the method, including: (i) when it is equivalent to lasso and when to ridge regression; (ii) main advantage(s) and disadvantage(s).

b (8 pt.)

Describe the relationship between Elastic Net and Bridge regression,

$$\hat{\beta}_{\text{Bridge}} = \underset{\beta}{\operatorname{argmin}} \left\{ \sum_{i=1}^N \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2 + \lambda \sum_{j=1}^p |\beta_j|^q \right\}$$

highlighting their similarities and differences in the case $1 \leq q \leq 2$. In addition, provide two specific cases in which the two methods are completely equivalent.

(Continued on page 2.)

Problem 2 Random Forests

Consider $Z = (y, X)$, where y is an N -dimensional vector of response and X an $N \times p$ input matrix, and B a large number of iterations. An early version of random forest (Ho, 1998) can be described by the following algorithm:

- For $b = 1, \dots, B$,
 1. generate a bootstrap sample $Z^* = (y^*, X^*)$;
 2. select $m < p$ variables;
 3. fit a statistical tree on (y^*, X_m^*) , where X_m^* only contains the m variable selected in the point (2);
- aggregate the results to get the final estimate.

a (8 pt.)

Highlight the difference with Leo Breiman's version of random forest that became "the" random forest algorithm (and was presented in the course). In addition, consider the simulation example described in the caption of Figure 1 (edited version of Figure 15.7 from Hastie et al, 2009). Do you expect that the algorithm above would have performed better, worse or equally good than Breiman's version? Why?

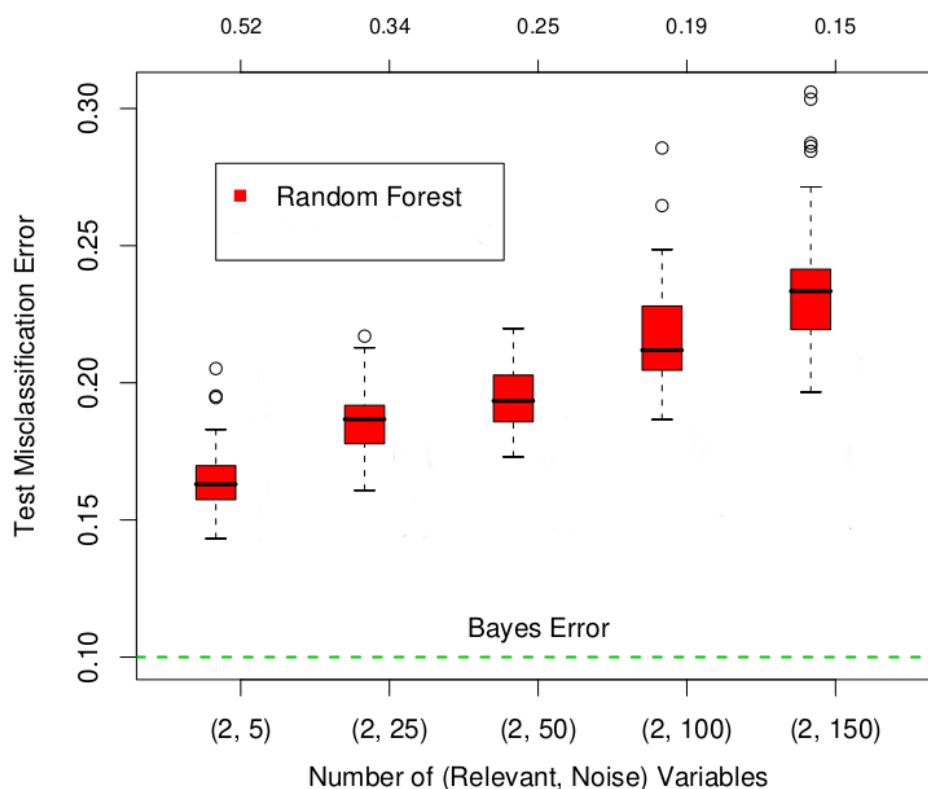


Figure 1: Results over 50 simulations (each time a training set of 300 observations has been generated, with a test set of 500 observations). Here the response depends on two variables, and an increasing number of noise variables are added (see the x-axis). The default $m = \sqrt{p}$ has been used, where p is the total number of variables and m the number of selected ones.

(Continued on page 3.)

b (8 pt.)

Consider the bias of a (Breiman's) random forest (regression) estimator. Hastie et al (2009) argue that it

“is the same as the bias of any of the individual sample trees”

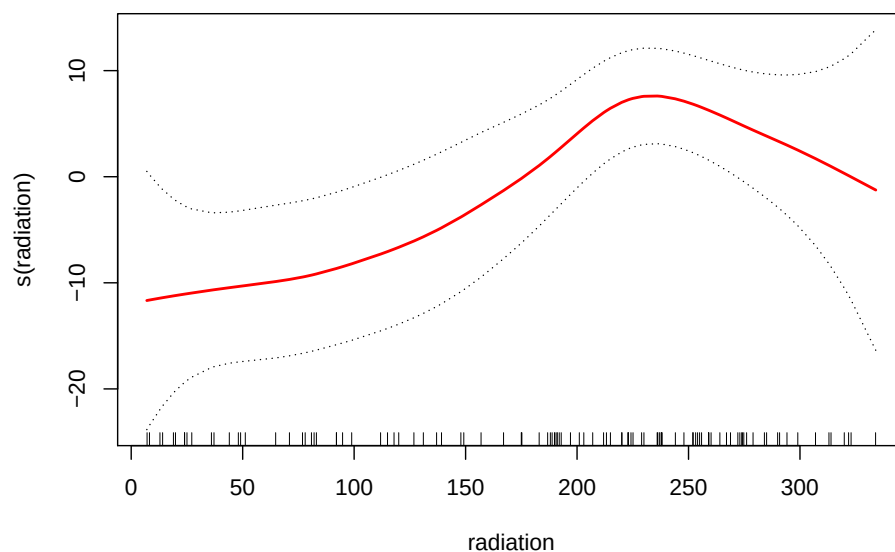
and it

“is also typically greater (in absolute terms) than the bias of an unpruned tree grown to Z , since the randomization and reduced sample space impose restrictions”.

Explain why a random forest bias is the same of any individual sample tree and why randomization and reduced sample space cause larger bias.

Problem 3 Splines

The effect of the solar radiation (in Ly) on the ozone concentration (in ppb) in the city of New York has been modelled with a spline in an additive model, obtaining the following result:



(data from Hastie et al, 2009)

a (8 pt.)

Describe the effect of the solar radiation on the ozone concentration, highlighting the differences with what one would have obtained in the case of a classical linear regression model.

(Continued on page 4.)

b (8 pt.)

Argue that the natural cubic spline $g(x)$ with knots at each of the x_i , with $a < x_1 < \dots < x_N < b$, $N \geq 2$, is the solution of

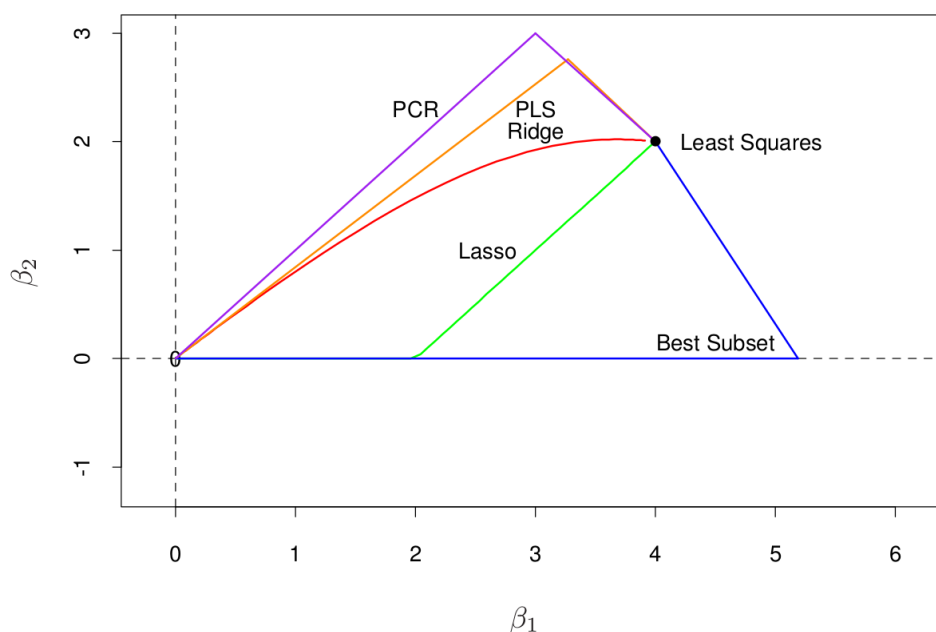
$$\operatorname{argmin}_f \left\{ \sum_{i=1}^N (y_i - f(x_i))^2 + \lambda \int_a^b f''(t)^2 dt \right\}.$$

To do so, first show that $\int_a^b g''(t)^2 dt \leq \int_a^b \tilde{g}''(t)^2 dt$ for any other differentiable function $\tilde{g}(x)$ that interpolates the x_i . You may want to use the fact that $\int_a^b g''(t)(\tilde{g}(t)'' - g(t)'') dt = 0$.

Problem 4 Comparison among methods

Consider the following figure from Hastie et al. (2009),

$$\rho = 0.5$$



where “PCR” stands for “principal component regression”, “PLS” for “partial least square” and ρ is the correlation between X_1 and X_2 , whose regression coefficients are β_1 and β_2 , respectively. Here the point “Least Squares” denotes $\beta^{LS} = (\beta_1^{LS}, \beta_2^{LS})$, i.e., the point of the parametric space obtained by ordinary least squares.

a (12 pt.)

Describe the figure above, focusing, in particular on:

- the relationship between PCR and Ridge regression, explaining what makes their path similar and what makes it different;

(Continued on page 5.)

- the behaviour of the Lasso's path, both explaining why it changes direction in $(\beta_1, \beta_2) = (2, 0)$ and pointing out the values (if they can be identified) when $\lambda = 0$ and $\lambda = \infty$;
- why Best Subset reaches the point $(\beta_1, \beta_2) = (5.2, 0)$ even if the first component of $\beta^{LS} = (\beta_1^{LS}, \beta_2^{LS})$ is smaller than 5.2 (specifically, 4).

b (8 pt.)

Add the path of a component-wise boosting algorithm with linear base learners to the figure. Describe the reasons behind your choice.

Problem 5 Boosting

Consider a gradient boosting algorithm with a linear base learner, in a regression setting, $y = f(x) + \epsilon$, where $E[\epsilon] = 0$, and a boosting algorithm in the form

-
1. initialize the estimate, e.g., $\hat{f}^{[0]}(x) \equiv 0$;
 2. for $m = 1, \dots, m_{\text{stop}}$,
 - compute the negative gradient vector, $u(y, f(x)) = -\frac{\partial L(y, f(x))}{\partial f(x)}$;
 - fit the base learner to the negative gradient vector, $\hat{h}(u(y, \hat{f}^{[m-1]}), x)$;
 - update the estimate, $\hat{f}^{[m]}(x) = \hat{f}^{[m-1]} + \nu \hat{h}(u, x)$;
 3. return the final estimate $\hat{f}^{[m_{\text{stop}}]}(x)$.
-

where $L(y, f(x))$ is the loss function and $h(\cdot)$ a base-learner.

a (8 pt.)

Briefly describe the concept of bias-variance trade-off and relate the choice of the tuning parameters m_{stop} and ν to it.

b (8 pt.)

Consider the case in which we have a count as the response and we want to use a Poisson negative log-likelihood as a loss function. Given that the density of a Poisson random variable is

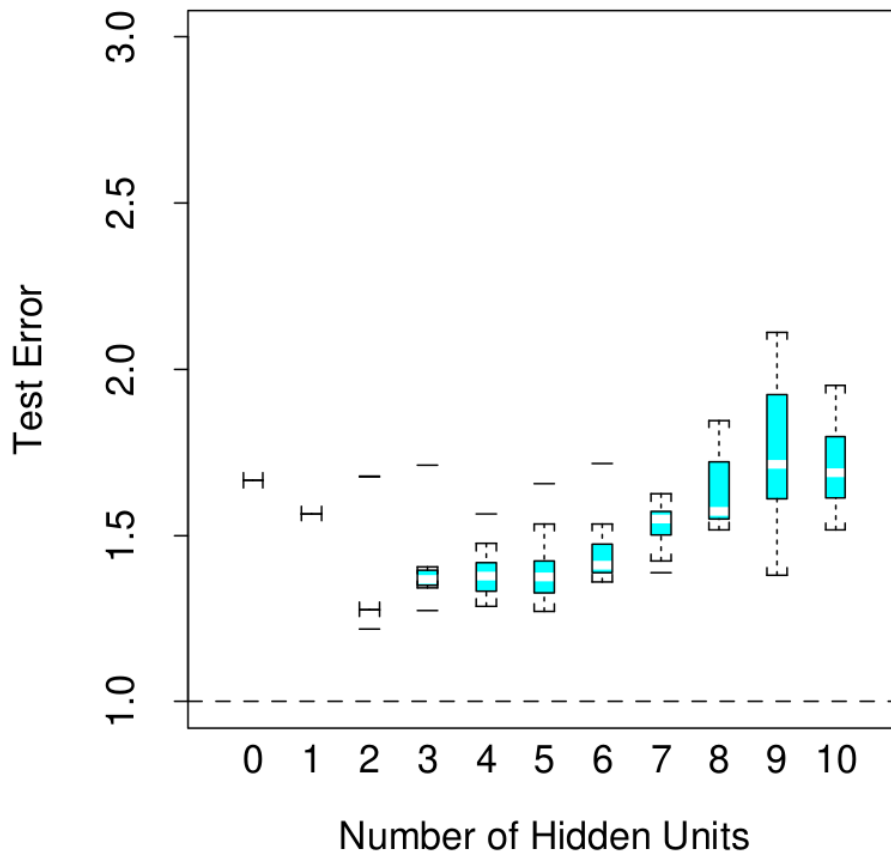
$$p(y) = \frac{e^{-\lambda(x)} \lambda(x)^y}{y!}$$

and that we may want to use a canonical link function, i.e., $\lambda(x) = e^{f(x)}$, derive the negative gradient $u(y, f(x))$.

(Continued on page 6.)

Problem 6 Neural Networks

Consider the following figure (Hastie et al, 2009, left panel of Figure 11.6),



where the results of a simulation study are shown. The test errors are computed for Neural Networks with different numbers of Hidden Units (x-axis), each for 10 different choices of starting weights. The Neural Networks have only one hidden layer and a fixed weight decay parameter of 5×10^{-4} .

a (8 pt.)

Despite the general suggestion of using a large number of Hidden Units, here the performance clearly worsen when increasing this parameter. Explain why this happens and what can be done to solve the issue.

b (8 pt.)

Relate the size of the Neural Network weights to the (model) complexity.

Bibliography

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*. Springer.

Ho, T. K. (1998). The random subspace method for constructing decision forests. *IEEE Trans. Pattern Anal. Mach. Intell.*, 20: 832-844.

THE END

UNIVERSITY OF OSLO

Faculty of mathematics and natural sciences

Exam in: STK-IN4300/STK-IN9300 — Statistical learning methods in Data Science

Day of examination: Monday, November 25th, 2020

Examination hours: 9.00 – 13.00

This problem set consists of 4 pages.

Appendices: None.

Permitted aids: None.

Please make sure that your copy of the problem set is complete before you attempt to answer anything.

Problem 1 Penalized regression

Consider data simulated with the following setting:

- $\beta_i \sim N(0, 2)$, $i = 1, \dots, p$;
- $X \sim N_p(\underline{0}, \Sigma)$, where: (i) $N_p(\cdot, \cdot)$ denotes a p -dimensional multivariate Gaussian distribution; (ii) $\underline{0}$ is a p -dimensional vector of 0; (iii) Σ is a $p \times p$ matrix with diagonal elements equal to 1 and all other elements equal to 0.9;
- $y = X\beta + \epsilon$, with $\beta = (\beta_1, \dots, \beta_p)^T$ and $\epsilon \sim N(0, 1)$.

a (7 pt.)

If you were forced to choose between ridge regression and lasso, which one would you have used to predict y on a test set generated with the same setting? Why?

b (7 pt.)

Would your choice have been the same if you ignored the first information on β ? Why?

c (6 pt.)

Do you think that elastic net could have been a better choice in the situation of point (b)? Why?

(Continued on page 2.)

Problem 2 Hjort-Glad estimator

Consider the Hjort-Glad estimator for density estimation,

$$\hat{f}_{HG}(x) = \frac{1}{N} \sum_{i=1}^N K_{\lambda}(x_i - x) \frac{f_0(x, \hat{\theta})}{f_0(x_i, \hat{\theta})}.$$

a (10 pt.)

Explain the logic behind the construction of such estimator, clarifying the role of the two terms $f_0(x, \hat{\theta})$ and $\sum_{i=1}^N \frac{K_{\lambda}(x_i - x)}{f_0(x_i, \hat{\theta})}$.

b (10 pt.)

Imagine you have to estimate the density of the variable “quantity of wine drunk by a person in a year”: Which distribution would you use in $f_0(x, \hat{\theta})$? Explain what kind of problem (that one can face using a classical non-parametric density estimator) your choice can solve.

Problem 3 Cross-validation

Consider the following situation: we want to evaluate the predicting performance of a lasso procedure, but we do not have enough data to split them. Therefore, we decide to use cross-validation and we proceed as follows:

- we implement a K-fold cross-validation procedure to identify the best tuning parameter λ ;
- we implement leave-one-out cross-validation to estimate the prediction error: the lasso with the estimated λ is trained on $n-1$ folds and evaluated on the remaining observation. The procedure is repeated n times, in which each observation plays, in turn, the role of the test observation, and the results averaged.

a (10 pt.)

Explain why the above procedure is wrong and provide a valid alternative.

b (10 pt.)

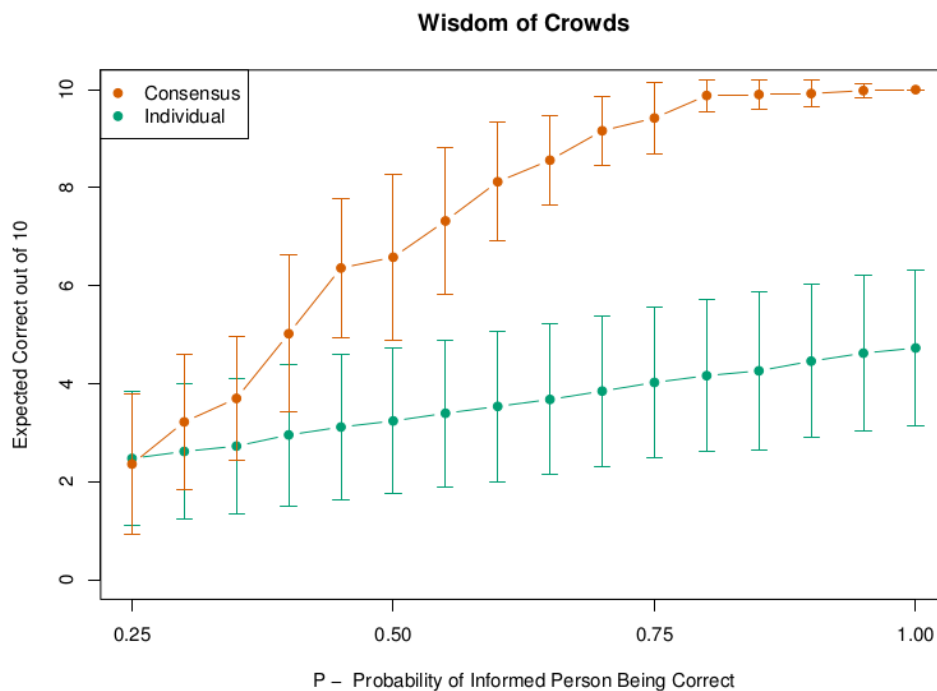
Describe the choice of the number of folds in a cross-validation procedure in terms of bias-variance trade-off.

(Continued on page 3.)

Problem 4 Wisdom of the Crowd and Bagging

Consider the following example from “The Elements of Statistical Learning” of Hastie et al (2009, Figure 8.11):

“Simulated academy awards voting. 50 members vote in 10 categories, each with 4 nominations. For any category, only 15 voters have some knowledge, represented by their probability of selecting the “correct” candidate in that category (so $P = 0.25$ means they have no knowledge). For each category, the 15 experts are chosen at random from the 50. Results [reported in the figure below] show the expected correct (based on 50 simulations) for the consensus, as well as for the individuals. The error bars indicate one standard deviation.”



a (7 pt.)

How do we expect the two curves behave on the left of the interval considered in the plot (i.e., when $P \in [0; 0.25]$)? What does it mean for bagging?

b (6 pt.)

Compute the value of the green curve at $p = 0$.

c (7 pt.)

Derive the formula of the variance for B identically distributed random variables,

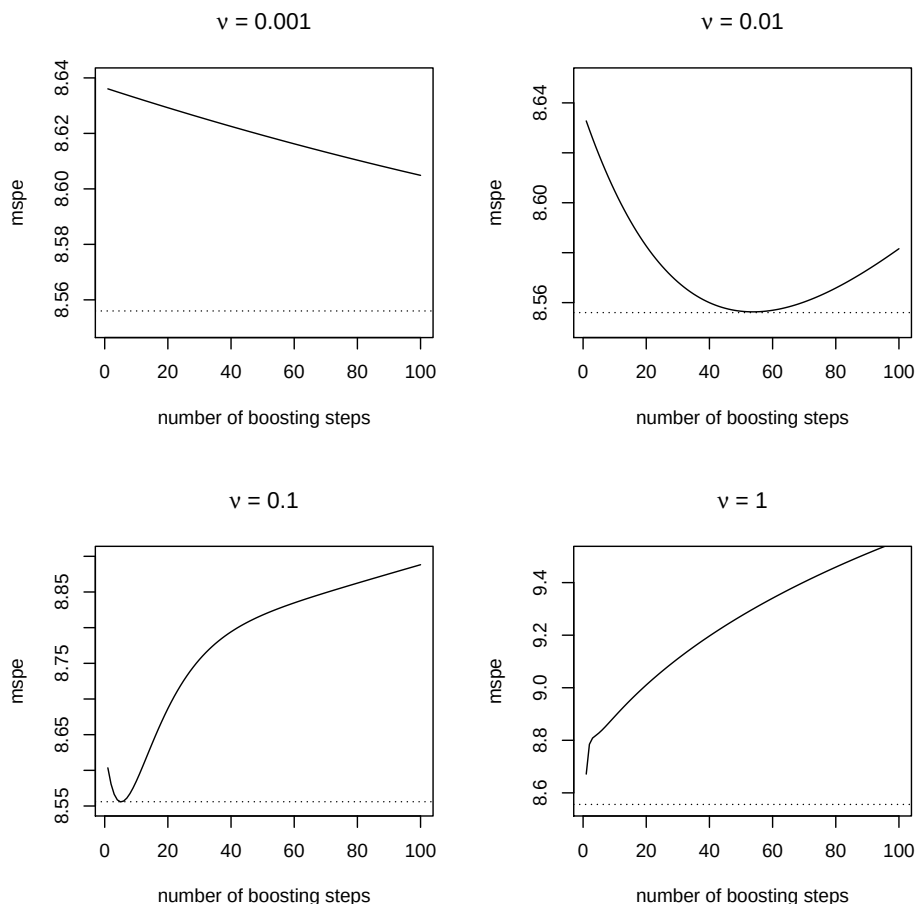
$$\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2,$$

and explain how bagging acts in order to improve the prediction performance of a single tree, and how this can be further improved.

(Continued on page 4.)

Problem 5 Boosting

Consider the following figure, obtained by using the R package `mboost`. Here the mean square prediction error has been computed as a function of the number of boosting steps, for four component-wise boosting models, each with a different value of the boosting step size ν .



a (10 pt.)

Explain the reasons for which we expected such a behaviour for the four curves, choose the best values for the tuning parameters in this case and justify your choice.

b (10 pt.)

Describe what differentiates the component-wise version of boosting from the standard one and describe two of its advantages.

THE END

UNIVERSITY OF OSLO

Faculty of mathematics and natural sciences

Exam in: STK-IN4300/STK-IN9300 — Statistical learning methods in Data Science

Day of examination: Thursday, December 5th, 2019

Examination hours: 14.30–18.30

This problem set consists of 4 pages.

Appendices: None.

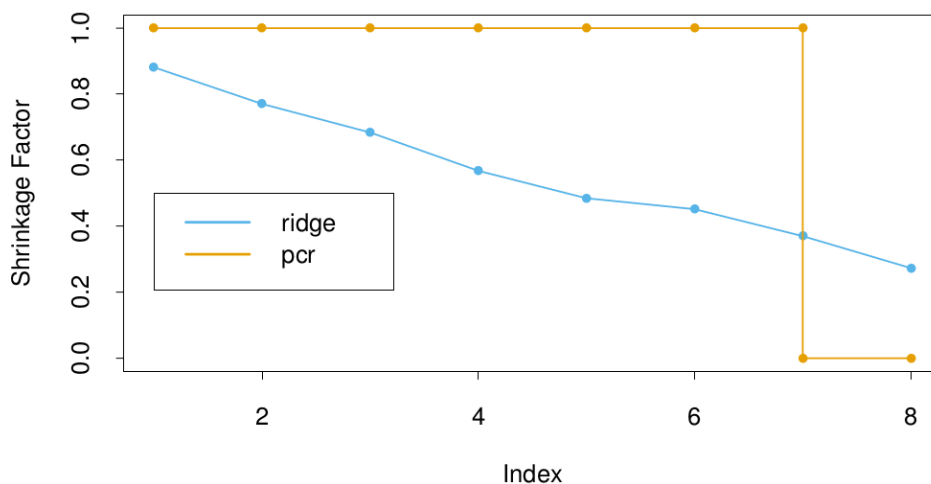
Permitted aids: None.

Please make sure that your copy of the problem set is complete before you attempt to answer anything.

Problem 1 Penalized regression

a Ridge versus principal component regression (10 pt.)

Consider the following figure from the textbook (Hastie, Tibshirani & Friedman, 2009, *The Elements of Statistical Learning*, Figure 3.17), where the x -label “Index” denotes the index of the principal components:



Explain the figure above, highlighting the differences between *ridge regression* and *principal component regression* when it concerns their shrinkage effect.

(Continued on page 2.)

b Sparse group lasso

Consider the following version of lasso, called *sparse group lasso*,

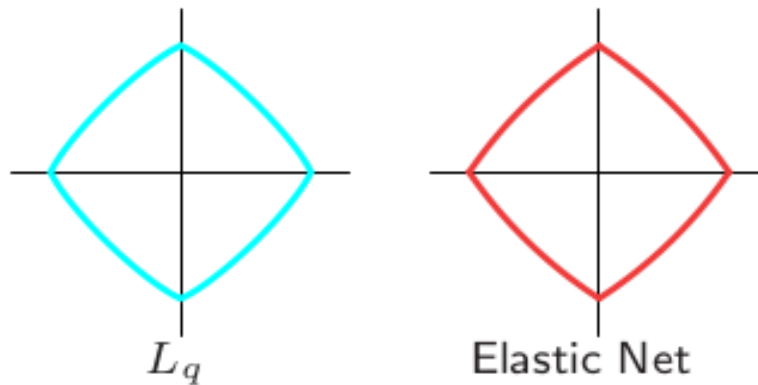
$$\min_{\beta} \left\{ \left\| y - \beta_0 \vec{1} - \sum_{\ell=1}^L X_{\ell} \beta_{\ell} \right\|_2^2 + (1 - \alpha) \lambda \sum_{\ell=1}^L \sqrt{p_{\ell}} \|\beta_{\ell}\|_2 + \alpha \lambda \|\beta\|_1 \right\},$$

where $\vec{1}$ denotes an N -dimensional vector of 1s, $\lambda \geq 0$ and $0 \leq \alpha \leq 1$. Answer to the following questions:

- Why does β_0 appears only in the first term? **(3 pt.)**
- What does it happen when $\alpha = 0$ and $\alpha = 1$, respectively? **(2 pt.)**
- Briefly describe the concept of “bet on sparsity”. **(5 pt.)**

c Elastic net versus bridge regression (10 pt.)

Briefly describe *elastic net* and *bridge regression*, and explain why, despite the corresponding constraints are almost indistinguishable in the figure here below (Hastie, Tibshirani & Friedman, 2009, The Elements of Statistical Learning, Figure 3.13), they provide, in general, quite different models.

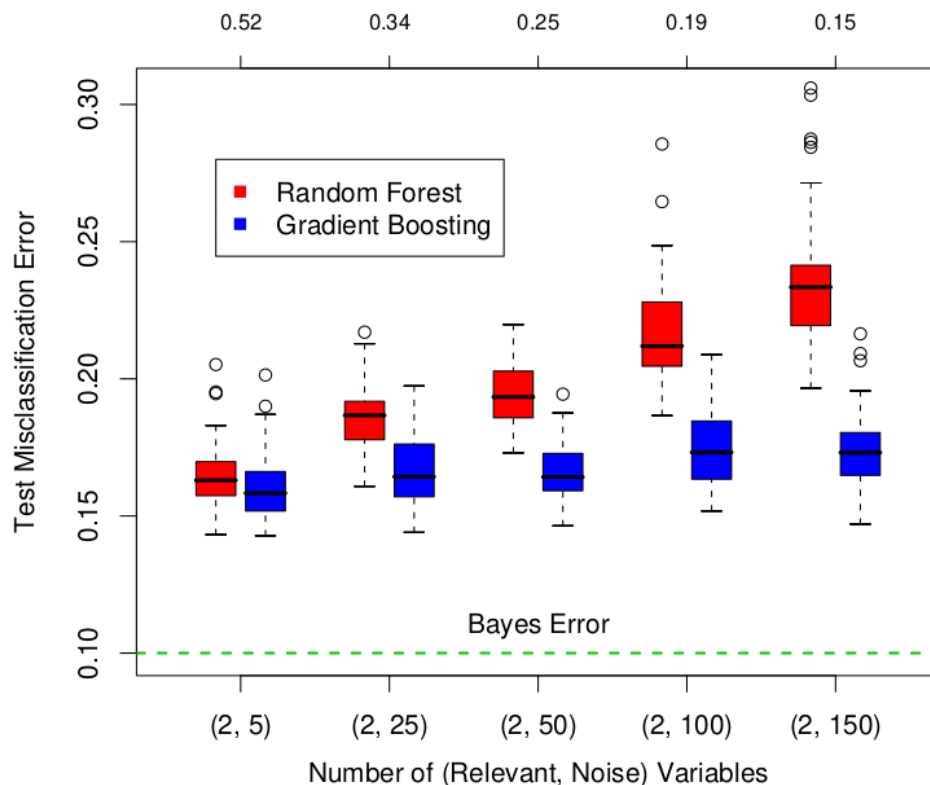
**Problem 2 Ensemble Methods****a Bagging (10 pt.)**

Consider a classification problem and how to aggregate the results of the single trees in a bagging classifier. The aggregation can be done by looking at the estimated classes or at the class-probability estimates. Show with a simple example that the two procedures can produce different results in terms of classification of an observation.

b Random Forests (10 pt.)

Consider the figure below (Hastie, Tibshirani & Friedman, 2009, The Elements of Statistical Learning, Figure 15.7),

(Continued on page 3.)



in which the results over 50 simulations (each time a training set of 300 observations has been generated, together with a test set of 500 observations) for *random forests* (red box-plot) and *gradient boosting* (blue box-plots, not relevant for the exercise) have been reported. Here the true boundary (it is a binary classification problem) depends on two variables, and an increasing number of noise variables are added (see x-axis). The default $m = \sqrt{p}$, where m is the number of candidate variables randomly selected as input before each split in the trees and p is the total number of variables, has been used. Explain why the performance of *random forests* worsen with p increasing.

c Boosting 1 (10 pt.)

Consider the regression model $y_i = f(x_i) + \epsilon_i, i = 1, \dots, N$, where ϵ_i are i.i.d. random variables with $E[\epsilon_i] = 0$ and $\text{Var}[\epsilon_i] = \sigma^2$. Bühlmann & Yu (2003, Journal of the American Statistical Society) showed that, for L_2 Boost, if

$$\frac{\mu_k^2}{\sigma^2} > \frac{1}{(1 - \lambda_k)^2} - 1 \quad (1)$$

for all k with $\lambda_k < 1$, then $\text{MSE}_{\mathcal{B}_m} < \text{MSE}_{\mathcal{S}}$, where $\text{MSE}_{\mathcal{B}_m}$ and $\text{MSE}_{\mathcal{S}}$ denote the mean square errors obtained using the boosting operator and the corresponding linear operator $\mathcal{S}$ used as base learner, respectively. Here, λ_k is the k -th eigenvalue of $\mathcal{S}$ and μ_k represents the true regression function corresponding to the k -th eigenvector of $\mathcal{S}$.

Interpret Equation (1), focusing on the importance of “shrinkage” in boosting.

(Continued on page 4.)

d Boosting 2

Consider the following component-wise gradient boosting algorithm,

-
1. initialize the estimate, e.g., $\hat{f}_j^{[0]}(x) \equiv 0, j = 1, \dots, p$;
 2. for $m = 1, \dots, m_{\text{stop}}$,
 - compute the negative gradient vector, $u = - \left. \frac{\partial L(y, f(x))}{\partial f(x)} \right|_{f(x)=\hat{f}^{[m-1]}(x)}$;
 - $\forall j$, fit the base learner to the negative gradient vector, $\hat{h}(u, x_j)$;
 - select the best update j^* ;
 - update the estimate, $\hat{f}_{j^*}^{[m]}(x) = \hat{f}_{j^*}^{[m-1]} + \nu \hat{h}(u, x_{j^*})$;
 3. final estimate, $\hat{f}_{m_{\text{stop}}}(x) = \sum_{j=1}^p \hat{f}_j^{[m_{\text{stop}}]}(x)$.
-

where $L(y, f(x))$ is a generic loss function and $h(\cdot)$ a base-learner:

- Identify the tuning parameters of the algorithm; **(2 pt.)**
- Describe how they are usually computed in practice; **(2 pt.)**
- Relate them to the prediction performance of the final model; **(3 pt.)**
- Explain why their optimal values are related to each other. **(3 pt.)**

Problem 3 Bias-variance trade-off**a Expected prediction error (10 pt.)**

Consider $y = f(x) + \epsilon$, with $E[\epsilon] = 0$ and $\text{Var}[\epsilon] = \sigma_\epsilon$. Show mathematically that, in the case of squared-error loss, the expected prediction error of a regression fit $\hat{f}(x)$ at an input point $x = x_0$ can be decomposed into: irreducible error, squared bias, variance. Moreover, briefly explain what these three terms are.

b Boosting (10 pt.)

Consider the model $y_i = f(x_i) + \epsilon_i$, $E[\epsilon_i] = 0$, $\text{Var}[\epsilon_i] = \sigma$, $i = 1 \dots, N$. Derive the formula of the squared bias for L₂Boost,

$$\text{bias}(m, \mathcal{S}; f)^2 = N^{-1} f^T U \text{diag}((1 - \lambda_k)^{2m+2}) U^T f,$$

when a symmetric learner $\mathcal{S}$, with eigenvalues λ_k and eigenvectors building the columns of the orthonormal matrix U , is used. Here f denotes the vector of the true regression function and m the number of boosting steps.

Hint: remember that the L₂Boost operator $\mathcal{B}_m$ can be rewritten as $\mathcal{B}_m = U D_m U^T$, with $D_m = \text{diag}(1 - (1 - \lambda_k)^{m+1})$ and $U U^T = U^T U = I$.

c Model complexity (10 pt.)

Relate the concept of model complexity to the concept of bias-variance trade-off, and show how this works for the k -nearest neighbours algorithm.

THE END

UNIVERSITY OF OSLO

Faculty of mathematics and natural sciences

Exam in: STK-IN4300/STK-IN9300 — Statistical learning methods in Data Science

Day of examination: Monday, December 17th, 2018

Examination hours: 14.30 – 18.30

This problem set consists of 4 pages.

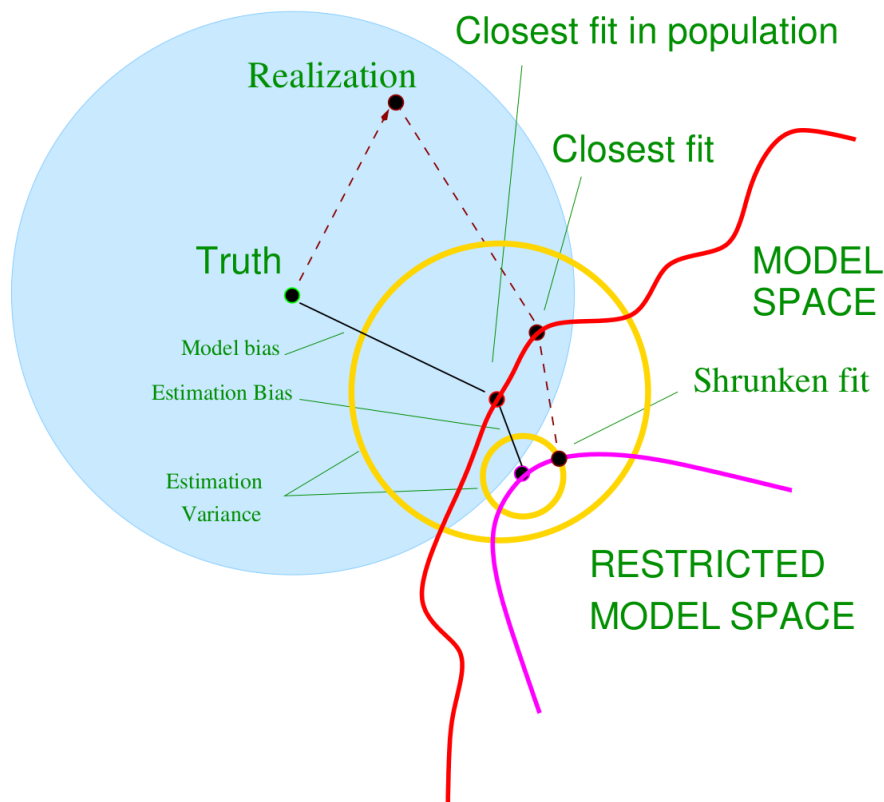
Appendices: None.

Permitted aids: None.

Please make sure that your copy of the problem set is complete before you attempt to answer anything.

Problem 1 Penalized regression

Consider the following figure from the text book (Hastie, Tibshirani & Friedman, 2009, The Elements of Statistical Learning):



a

Using the figure above, explain the concept of bias-variance trade-off.

(Continued on page 2.)

b

Show analytically the same concept of the point above by mathematically comparing bias and variance of the ordinary least square estimator and of the ridge estimator.

Problem 2 Bootstrapping for model evaluation

Consider the following procedure to estimate the prediction error:

1. generate B bootstrap samples $z_1, \dots, z_B$, where $z_b = \{(y_1^*, x_1^*), \dots, (y_N^*, x_N^*)\}$, $b = 1, \dots, B$ and (y_i^*, x_i^*) , $i = 1, \dots, N$, is an observation sampled from the original dataset;
2. apply the prediction rule to each bootstrap sample to derive the predictions $\hat{f}_b^*(x_i)$, $b = 1, \dots, B$;
3. compute the error for each point, and take the average,

$$\widehat{\text{Err}}_{\text{boot}} = \frac{1}{B} \sum_{b=1}^B \frac{1}{N} \sum_{i=1}^N L(y_i, \hat{f}_b^*(x_i)).$$

a

Explain why this procedure is incorrect and suggest a different way to proceed which still uses a bootstrap approach.

b

Describe the `0.632 bootstrap` and the `0.632+ bootstrap` procedures, explaining in particular the rationale behind their construction.

Problem 3 Smoothing splines

Consider the following problem: among all functions $f(x)$ with two continuous derivatives, find one that minimizes the penalized residual sum of squares

$$RSS(f, \lambda) = \sum_{i=1}^N (y_i - f(x_i))^2 + \lambda \int \{f''(t)\}^2 dt \quad (1)$$

where $\lambda \geq 0$.

a

Define the role of the penalization term $\lambda \int \{f''(t)\}^2 dt$ in relation to its specific form, and discuss what happens when the smoothing parameter λ varies.

(Continued on page 3.)

b

The solution of minimization problem (1) can be written as a natural spline

$$f(x) = \sum_{j=1}^N N_j(x) \theta_j.$$

Rewrite (1) as a function of θ (i.e., $RSS(\theta, \lambda)$) and use its solution to show that a smoothing spline for a fixed λ is a linear smoother (linear operator). Use it to define the effective degrees of freedom of a smoothing spline.

Problem 4 Bagging

a

Describe bagging, mentioning at least one advantage with respect to a single tree and a disadvantage with respect to a boosted tree model.

b

Consider a classification problem and how to aggregate the results of the single trees in a bagging classifier. The aggregation can be done by looking at the estimated classes or at the class-probability estimates. Show with a simple example that the two procedures can produce different results in terms of classification of an observation.

Problem 5 Boosting

a

Show that the additive expansion produced by AdaBoost is estimating one-half the log-odds of $P(Y = 1|X = x)$, where Y is the binary response and X the input matrix.

b

Consider the following algorithm,

1. initialize the estimate, e.g., $f_0(x) = 0$;
2. for $m = 1, \dots, m_{\text{stop}}$,
 - 2.1 compute the negative gradient vector, $u_m = \text{OMITTED}$;
 - 2.2 fit the base learner to the negative gradient vector, $h_m(u_m, x)$;
 - 2.3 update the estimate, $f_m(x) = f_{m-1}(x) + \nu h_m(u_m, x)$.

(Continued on page 4.)

3. final estimate, $\hat{f}_{m_{\text{stop}}}(x) = \sum_{m=1}^{m_{\text{stop}}} \nu h_m(u_m, x)$

Name the specific boosting algorithm and write the complete formula in point 2.1 (where the word **OMITTED** is) for a generic loss function $L(y, f(x))$.

THE END

UNIVERSITY OF OSLO

Faculty of mathematics and natural sciences

Exam in: STK-IN4300/9300 — Statistical Learning
Methods in Data Science

Day of examination: Tuesday, 28 November 2023

Examination hours: 15.00 – 19.00

This solution proposal consists of 5 pages.

Appendices: None.

Permitted aids: None.

Solution Proposal

Problem 1

(a) For the purpose of estimating the model parameters θ :

- (i) The least squares method selects the parameter values that minimizes the residual sum-of-squares (RSS):

$$\theta_{\text{ls}} = \underset{\theta}{\operatorname{argmin}} \left\{ \sum_{i=1}^N (y_i - f_{\theta}(x_i))^2 \right\}$$

- (ii) The maximum likelihood method selects the parameter values that maximizes the (log-)likelihood:

$$\theta_{\text{mle}} = \underset{\theta}{\operatorname{argmax}} \left\{ \sum_{i=1}^N \log[p(y_i | x_i; \theta, \sigma^2)] \right\},$$

where $p(Y | x; \theta, \sigma^2)$ is the conditional density function of $Y | x$.

- (b) Under the given assumptions of an additive error term $N(0, \sigma^2)$, we have that

$$(Y | x) \sim N(f_{\theta}(x), \sigma^2),$$

and the log-likelihood can thus be written as

$$\begin{aligned} L(\theta) &= \sum_{i=1}^N \left(-\frac{1}{2} \log[2\pi] - \log[\sigma] - \frac{1}{2\sigma^2} (y_i - f_{\theta}(x_i))^2 \right) \\ &= -\frac{N}{2} \log[2\pi] - N \log[\sigma] - \frac{1}{2\sigma^2} \sum_{i=1}^N (y_i - f_{\theta}(x_i))^2, \end{aligned}$$

where the only term involving θ is the last one, which is RSS up to a scalar negative factor. Thus, maximizing the log-likelihood w.r.t. θ is equivalent to minimizing the RSS.

(Continued on page 2.)

Problem 2

- (a) Under the given assumptions, we can decompose the squared-error loss as:

$$\begin{aligned}\mathbb{E}[(Y - \hat{f}(x_0))^2 | x_0] &= \mathbb{E}[(f(x_0) + \epsilon - \hat{f}(x_0))^2] \\ &= \mathbb{E}[f(x_0)^2] + \mathbb{E}[f(x_0)\epsilon] - \mathbb{E}[f(x_0)\hat{f}(x_0)] \\ &\quad + \mathbb{E}[\epsilon f(x_0)] + \mathbb{E}[\epsilon^2] - \mathbb{E}[\epsilon \hat{f}(x_0)] \\ &\quad - \mathbb{E}[\hat{f}(x_0)f(x_0)] - \mathbb{E}[\hat{f}(x_0)\epsilon] + \mathbb{E}[\hat{f}(x_0)^2].\end{aligned}$$

Now, since $f(x_0)$ is fixed (i.e. a constant) and $\mathbb{E}[\epsilon] = 0$, we have that

$$\begin{aligned}\mathbb{E}[f(x_0)\epsilon] &= f(x_0)\mathbb{E}[\epsilon] = 0, \\ \mathbb{E}[\epsilon^2] &= \text{Var}[\epsilon] = \sigma_\epsilon^2, \\ \mathbb{E}[\hat{f}(x_0)\epsilon] &= \mathbb{E}[\hat{f}(x_0)]\mathbb{E}[\epsilon] = 0,\end{aligned}$$

where the last equality use the fact that $\hat{f}(x_0)$ and ϵ are independent. We thus have that

$$\begin{aligned}\mathbb{E}[(Y - \hat{f}(x_0))^2 | x_0] &= f(x_0)^2 - 2f(x_0)\mathbb{E}[\hat{f}(x_0)] + \sigma_\epsilon^2 + \mathbb{E}[\hat{f}(x_0)^2] \\ &= \sigma_\epsilon^2 + (\mathbb{E}[\hat{f}(x_0)] - f(x_0))^2 - \mathbb{E}[\hat{f}(x_0)]^2 + \mathbb{E}[\hat{f}(x_0)^2] \\ &= \sigma_\epsilon^2 + (\mathbb{E}[\hat{f}(x_0)] - f(x_0))^2 + \mathbb{E}[(\hat{f}(x_0) - \mathbb{E}[\hat{f}(x_0)])^2],\end{aligned}$$

where the last step makes use of the variance formula:

$$\text{Var}[\hat{f}(x_0)] = \mathbb{E}[\hat{f}(x_0)^2] - \mathbb{E}[\hat{f}(x_0)]^2.$$

The first term is the variance of target around its true mean $f(x_0)$ (irreducible error), the second term represents the systematic error, that is, how much the average of our estimate differs from the true mean (squared bias), and the third term is the variance of our estimator, that is, how much it varies around its mean.

- (b) Here, k denotes the number of neighbors and it is inversely related to model complexity. From small values of k , the model is more flexible/complex and can better adapt to the true function $f(x)$. For larger values of k , the model will become less flexible/complex, since we will compute the average based on training points further away from x_0 . In terms of the bias-variance tradeoff, small values of k will give a model with low bias but high variance and large values will give a model with high bias but low variance.

Problem 3

- (a) The general idea behind the considered shrinkage methods is to add a regularization term to the least squares objective that will shrink the estimates towards zero in relation to the least-squares solution. The key idea is that by adding bias to the model through shrinkage, one is able to reduce the variance and thus (possibly) decrease the

(Continued on page 3.)

expected prediction error. The amount of shrinkage is controlled by a regularization parameter, often denoted by λ , which in this case is directly (inversely) related to t in the given optimization problem.

- (b) (i) The key difference between ridge regression and lasso, in terms of the model they produce, is that lasso encourage sparse solutions by shrinking some parameters all they way to zero (built-in feature selection). This property is reflected in the constraint region of lasso in Figure 1 (on the left), in the sense that one will typically hit one the "sharp corners" of the region, which represent solutions where one of the parameters is set to zero. This is in contrast to the constraint region of ridge regression (on the right) where there is no "sharp corners" and one will typically not have any zero parameter estimates.
- (ii) By increasing t (corresponds to decreasing λ), the constraint regions in the figure will increase in size and the parameters will thus be shrunken less (and vice versa). If we increase t enough, the constraint region will cover the least-squares solution and we will not get any shrinkage.

Problem 4

- (a) The problem with the described procedure is that initial screening in Step 1 is done prior to cross-validation, that is, it is using all the data. In other words, when training a model in Step 2 it has already used information from all data (including the fold that is used for testing). As a result, the averaged test error will be overly optimistic as an estimate of the expected test error. One way to improve the proposed procedure would be to move the screening step inside the cross-validation loop, such that the screening would only be based on the training data w.r.t. the current fold.
- (b) (i) This is the generalization error or expected test error given the training set. In terms of model evaluation, this is typically what we are interested in for a given application where we have some data on which to train the model, that is, how well will the model trained on the available data perform in predicting new observations.
- (ii) This is the expected test error when we do not fix the training data, but instead compute the expectation also w.r.t. the training data. This error is typically more convenient to work with from a statistical analysis point of view.
- (iii) This is the in-sample error where we keep the input values and compute the expected error assuming that only the output values are redrawn. The in-sample error is overly optimistic in terms of estimating the expected test error in the sense that the test and training inputs coincide. Estimates of the in-sample error can still be very useful for model selection. For example, the AIC and BIC are popular model selection criteria that can be viewed as estimates of the in-sample error.

(Continued on page 4.)

Problem 5

- (a) Bootstrap aggregating (or bagging) is a technique where: (i) B bootstrap samples are drawn by resampling the training data with replacement; (ii) a tree is fitted on each bootstrap sample; (iii) the predictions of the bootstrap trees are averaged to form the final prediction. Bagging is a variance reduction technique and can thus be particularly useful for improving the results of high-variance (and low-bias) methods, such as trees.
- (b) Since the trees are fitted on bootstrap samples from the same distribution, the predictions of the individual trees are typically highly correlated. Thus, while the second term on the right-hand side of the equation will disappear as B is increased, the first term will not be affected. This implies that the correlation between bootstrap trees will limit the benefits of bagging. The random forest improves variance reduction by reducing the correlation between trees. This is achieved by, at each splitting step during the tree growing process, randomly selecting a subset of candidate features to split on.
- (c) The total variance is the variance of the prediction of a tree at point x , which includes the sampling variability of the training data $\mathbf{Z}$ as well as the variability due to the random selection of split candidates. The first term on the right-hand side is the sampling variability of the random forest ensemble. The second term is the within- $\mathbf{Z}$ variance, which is the expected variance due to the randomization step when growing the trees.

Problem 6

- (a) We compute the derivative of the objective function w.r.t. $f(x)$ and set it to zero:

$$\begin{aligned} \frac{d}{df(x)} \mathbb{E}_{Y|x}[e^{Yf(x)}] &= \frac{d}{df(x)} \left(e^{-f(x)} p(Y = 1|x) + e^{f(x)} p(Y = -1|x) \right) \\ &= -e^{-f(x)} p(Y = 1|x) + e^{f(x)} p(Y = -1|x) \\ &\stackrel{\text{set}}{=} 0. \end{aligned}$$

Then, we solve for $f(x)$:

$$e^{2f(x)} = \frac{p(Y = 1|x)}{p(Y = -1|x)} \Rightarrow f(x) = \frac{1}{2} \log \left[\frac{p(Y = 1|x)}{p(Y = -1|x)} \right],$$

and we know that solution is a minimum since $\frac{d^2}{d[f(x)]^2} \mathbb{E}_{Y|x}[e^{Yf(x)}] > 0$.

Adaboost classifies based on the sign of the additive expansion it produces (Step 3 in Alg. 10.1). This makes sense given the above population minimizer, since it corresponds to predicting the class label with the highest conditional probability (Bayes classifier).

- (b) Under the considered setting, the classification rule implies that observations with a positive margin, $y_i f(x_i) > 0$, are classified

(Continued on page 5.)

correctly and observations with a negative margin, $y_i f(x_i) < 0$, are misclassified (and the decision boundary is defined by $f(x_i) = 0$). Thus, a reasonable loss criterion should penalize negative margins more heavily than positive margins. This is not the case for the squared error loss, making it a bad choice in the considered setting. Both the exponential loss and binomial deviance are strictly decreasing functions of the margin, penalizing increasingly negative margins more than they reward increasingly positive ones. The exponential loss increases very quickly (exponentially) as a negative margin value is further decreased, putting a lot of focus on observations with large negative margins. In comparison, the binomial deviance increases more moderately (almost linearly) as a negative margin value is further decreased, and it is thus less influenced by observations with large negative margins. As a result, the binomial deviance is more robust in noisy settings (with high Bayes error rate), and especially when there are misspecification of class labels in the training data.

Sketch of solutions

STK-IN4300/9300 2022

Problem 1 Bias-variance trade-off

1a L_2 -Boosting

In the first two points we see that the bias of the L_2 -boosting operator decays exponentially fast for increasing m , while the variance has an exponentially slow increase. This represents an advantageous trade-off, which is not common to all procedures seen in the course. Since the mean squared error (MSE) is the sum of the squared bias and variance terms, we see that for $m \rightarrow \infty$ the MSE tends to σ^2 , the noise level, implying that it is not convenient to go on until theoretical convergence (it would only lead to overfitting). Instead, we want to ensure an early stop to avoid overfitting and obtain a lower MSE. The third point provides us with a condition under which boosting the smoother effectively improves over using the smoothing operator alone. The left-hand side can be interpreted as a signal-to-noise ratio, and we need it to be "large enough" to ensure it is larger than the right-hand side; the right-hand side, instead, needs to be "small enough": for this to happen, we want the term $1/(1 - \lambda_k)^2$ to be as small as possible, which means that λ_k should be as small as possible (we are also under the condition $0 < \lambda_k \leq 1$). Since λ_k for $k = 1, \dots, N$ are the eigenvalues of the smoothing operator, this means that the smoothing operator needs to be weak, and/or shrunk, in order for boosting to improve over using the smoother alone. These results are demonstrated not in general, but for boosting with the L_2 -loss and symmetric linear operators as base learners.

1b k-Nearest Neighbours

In kNN, the number of neighbours k is the tuning parameter that controls the complexity of the model. We know that, in general, increasing the model complexity entails a decrease in the bias and an increase in the variance (and viceversa when decreasing the model complexity). We observe this trade-off in the case of kNN: a higher k means a less complex model, hence a larger bias (since we include in the average also points that are far from the point where we are estimating) but a smaller variance (underfit) - the extreme example is considering all points as neighbours, where the estimate is then just an average over the whole sample; a lower k means a more complex model, resulting in a smaller bias but larger variance - the extreme case is $k = 1$, which has a training error equal to 0 but is characterized by a very large variance (overfit) as we are "averaging" using one data point only.

1c Smoothing splines

A spline that is trimming down the hills and filling in the valleys is underfitting the data in high curvature regions. This means that there is too much bias, and the complexity of the model should be increased. On the other hand, increasing the spline complexity will result in a higher variance due to the bias-variance trade-off: this is why it is important to ensure that the spline does not follow

the data too closely, i.e. avoid overfitting which leads to poor generalization ability.

Problem 2 Ensemble methods

2a Bagging

Bagging is an ensemble method based on bootstrap samples. The procedure is as follows:

- Given the original training set Z , a number B of bootstrap samples $Z_1^*, \dots, Z_B^*$ is generated;
- On each of the bootstrap samples, a tree is fitted;
- The results are aggregated, either via probability or via consensus.

In a classification problem with classes $\{1, \dots, K\}$, aggregating via *consensus* means choosing the class that received the highest number of votes (i.e. the class that was most frequently selected by the trees):

$$\hat{G}(x) = \operatorname{argmax}_k q_k(x)$$

where $k \in \{1, \dots, K\}$, and q_k is the proportion of trees voting for class k . Aggregating by *probability*, instead, means computing the probability of each class as an average over the B bootstrap samples, and then choosing the class with the highest resulting probability:

$$\hat{G}(x) = \operatorname{argmax}_k \left(\frac{1}{B} \sum_{b=1}^B p_k^{[b]}(x) \right)$$

where $p_k^{[b]}$ is the probability assigned to class k by the b -th tree.

The strength of the method is that averaging over bootstrap samples leads to a reduction of the variance component of the expected prediction error, in particular when using high-variance classifiers such as trees. Under squared-error loss, averaging reduces the variance while leaving the bias unchanged:

$$\begin{aligned} E_{\mathcal{P}}[(Y - \hat{f}^*(x))^2] &= E_{\mathcal{P}}[(Y - f_{bag}(x) + f_{bag}(x) - \hat{f}^*(x))^2] \\ &= E_{\mathcal{P}}[(Y - f_{bag}(x))^2] + E_{\mathcal{P}}[(f_{bag}(x) - \hat{f}^*(x))^2] \\ &\geq E_{\mathcal{P}}[(Y - f_{bag}(x))^2]. \end{aligned}$$

This does not apply in case of classification with 0-1 loss, due to non-additivity of bias and variance. In this case, bagging makes better a good classifier, and worst a bad one.

The limitation of bagging is that the pairwise correlation between bootstrap trees limits the advantages of reducing the variance by averaging. This is why the trees should be less correlated as possible (ideally, independent), and using random forest brings improvement over bagging.

2b Random Forest

- The first term on the right-hand side is the variance of the random forest estimator:

$$\text{Var}_Z[E_{\Theta|Z}[T(x; \Theta(Z))]] = \text{Var}_Z \hat{f}_{rf}(x) = \rho(x) \sigma^2(x)$$

where $\rho(x)$ is the sampling correlation between any pair of trees used in the averaging, and $\sigma^2(x)$ is the sampling variance of any single randomly drawn tree. Clearly, this term increases for increasing correlation, which in turns increases for m increasing. The second term is the the "within- Z " variance, which comes as a result of the randomization, and increases as m decreases. The coexistence of these two terms is what makes the choice of m non-trivial, in fact the best practice would be to select m via cross-validation.

- The left panel of the figure shows the concept expressed in the previous answer: we see that the "within- Z " variance decreases as m increases. Nonetheless, the total variance is increasing: this is because the sampling variance of the random forest ensemble increases for increasing m , and dominates over the "within- Z " variance. The right panel show the random forest bias-variance trade-off: as m increases, the bias is reduced but the variance increases, leading to an increased mean squared error. It is interesting to note that the squared bias of the ensemble is larger than that of a single fully grown tree (horizontal line in the right plot), but the variance is reduced, especially for small values of m .

2c Boosting

Since

$$\hat{f}^{[m]}(x_i) = \sum_{k=1}^m \hat{\beta}^{[k]} \hat{g}^{[k]}(x_i),$$

at the m -th step the exponential loss function is

$$L(y, f(x)) = \sum_{i=1}^N \exp[-y_i \sum_{k=1}^m \beta^{[k]} g^{[k]}(x_i)]$$

and $\hat{\beta}$ and $\hat{g}$ can be found as

$$\begin{aligned} (\hat{\beta}, \hat{g}) &= \underset{(\beta, g)}{\text{argmin}} \left\{ \sum_{i=1}^N \exp \left[-y_i \sum_{k=1}^m \beta^{[k]} g^{[k]}(x_i) \right] \right\} \\ &= \underset{(\beta, g)}{\text{argmin}} \left\{ \sum_{i=1}^N \exp \left[-y_i \left(\sum_{k=1}^{m-1} \beta^{[k]} g^{[k]}(x_i) + \beta g(x_i) \right) \right] \right\} \\ &= \underset{(\beta, g)}{\text{argmin}} \left\{ \sum_{i=1}^N w_i^{[m-1]} \exp[-y_i \beta g(x_i)] \right\} \end{aligned}$$

where $w_i^{[m-1]} = \exp[-y_i \hat{f}^{[m-1]}]$ does not depend on β and g in the m -th step.

• Minimization with respect to g

$$\begin{aligned}
\hat{g}^{[m]} &= \operatorname{argmin}_g \left\{ \sum_{i=1}^N w_i^{[m-1]} \exp[-y_i \beta g(x_i)] \right\} \\
&= \operatorname{argmin}_g \left\{ \sum_{i:g(x_i)=y_i} w_i^{[m-1]} \exp[-\beta \cdot 1] + \sum_{i:g(x_i) \neq y_i} w_i^{[m-1]} \exp[-\beta \cdot (-1)] \right\} \\
&= \operatorname{argmin}_g \left\{ \sum_{i:g_i=y_i} w_i^{[m-1]} e^{-\beta} + \sum_{i:g_i \neq y_i} w_i^{[m-1]} e^{-\beta} - \sum_{i:g_i \neq y_i} w_i^{[m-1]} e^{-\beta} + \sum_{i:g_i \neq y_i} w_i^{[m-1]} e^{\beta} \right\} \\
&= \operatorname{argmin}_g \left\{ \sum_{i=1}^N w_i^{[m-1]} e^{-\beta} + (e^{\beta} - e^{-\beta}) \sum_{i:g_i \neq y_i} w_i^{[m-1]} \right\} \\
&= \operatorname{argmin}_g \left\{ \sum_{i=1}^N w_i^{[m-1]} e^{-\beta} + (e^{\beta} - e^{-\beta}) \sum_{i=1}^N \mathbb{1}(y_i \neq \hat{g}(x_i)) w_i^{[m-1]} \right\} \\
&= \operatorname{argmin}_g \left\{ \underbrace{\sum_{i=1}^N \mathbb{1}(y_i \neq \hat{g}(x_i)) w_i^{[m-1]}}_{\text{weighted misclassification error}} \right\}
\end{aligned}$$

• Minimization with respect to β

$$\begin{aligned}
\hat{\beta}^{[m]} &= \operatorname{argmin}_{\beta} \left\{ \sum_{i=1}^N w_i^{[m-1]} \exp[-y_i \beta g(x_i)] \right\} \\
&= \operatorname{argmin}_{\beta} \left\{ \underbrace{\sum_{i:g_i=y_i} w_i^{[m-1]} e^{-\beta} + \sum_{i:g_i \neq y_i} w_i^{[m-1]} e^{\beta}}_{\mathcal{L}} \right\}
\end{aligned}$$

We compute the partial derivative of $\mathcal{L}$ wrt β and set it to 0:

$$\frac{\partial \mathcal{L}}{\partial \beta} = - \sum_{i:g_i=y_i} w_i^{[m-1]} e^{-\beta} + \sum_{i:g_i \neq y_i} w_i^{[m-1]} e^{\beta} \stackrel{!}{=} 0$$

Multiplying and dividing by e^{β} , we obtain:

$$\begin{aligned}
& - \sum_{i:g_i=y_i} w_i^{[m-1]} + e^{2\hat{\beta}} \sum_{i:g_i \neq y_i} w_i^{[m-1]} \stackrel{!}{=} 0 \\
\iff & e^{2\hat{\beta}} \sum_{i:g_i \neq y_i} w_i^{[m-1]} = \sum_{i:g_i=y_i} w_i^{[m-1]} \\
\iff & e^{2\hat{\beta}} \sum_{i:g_i \neq y_i} w_i^{[m-1]} = \sum_{i=1}^N w_i^{[m-1]} - \sum_{i:g_i \neq y_i} w_i^{[m-1]}
\end{aligned}$$

$$\begin{aligned}
\Longleftrightarrow e^{2\hat{\beta}} &= \frac{\sum_{i=1}^N w_i^{[m-1]} - \sum_{i:g_i \neq y_i} w_i^{[m-1]}}{\sum_{i:g_i \neq y_i} w_i^{[m-1]}} = \frac{\frac{\sum_{i=1}^N w_i^{[m-1]}}{\sum_{i=1}^N w_i^{[m-1]}} - \frac{\sum_{i:g_i \neq y_i} w_i^{[m-1]}}{\sum_{i=1}^N w_i^{[m-1]}}}{\frac{\sum_{i:g_i \neq y_i} w_i^{[m-1]}}{\sum_{i=1}^N w_i^{[m-1]}}} \\
&= \frac{1 - \text{err}}{\text{err}} \quad \text{with } \text{err} = \frac{\sum_{i:g_i \neq y_i} w_i^{[m-1]}}{\sum_{i=1}^N w_i^{[m-1]}}
\end{aligned}$$

giving

$$\hat{\beta} = \frac{1}{2} \log\left(\frac{1 - \text{err}}{\text{err}}\right)$$

- If we treated the minimization of the loss function as a "standard" minimization problem to be solved by e.g. gradient descent or other numerical methods, we would likely overfit. In fact, our aim is minimizing the loss *on the new, unseen data*, but we only have the training set to work with. For this reason, we use boosting to ensure an early stop in the minimization process: we do not want to reach the actual minimum, but stop a bit before. This amounts to accepting some bias to reduce the variance, resulting in a lower prediction error.

Problem 3 Projection pursuit and neural networks

3a Projection pursuit

Projection pursuit is an additive model in derived features of the inputs,

$$f(X) = \sum_{m=1}^M g_m(\omega_m^T X) = \sum_{m=1}^M g_m(V_m),$$

where X is a p -dimensional input vector, $\omega_1, \dots, \omega_M$ are p -dimensional unit vectors and $V_m = \omega_m^T X$ are the derived features. V_m is the projection of X onto ω_m : the algorithm searches for the best ω_m to fit the data. The function $g_m(\omega_m^T X)$ is called *ridge function* and varies only in the direction of ω_m - hence, it is estimated together with ω_m in the algorithm, usually with splines or local regression. Projection pursuit can be considered a *universal approximator* because, under a suitable choice of g_m and for M large, it is successful in approximating any continuous function in $\mathbb{R}^p$. However, since the model is complex, interpretability is lost (with the exception of the case $M = 1$) and should be used as a pure-prediction method.

The connection between projection pursuit and neural networks resides in both being sums of non-linearly transformed linear models. Instead of the non-parametric functions g_m of projection pursuit, for neural network we have the *activation function* σ (very often a sigmoid function) which is multiplied with a weight β_m . The formal connection is:

$$\begin{aligned}
g_m(\omega_m^T X) &= \beta_m \sigma(\alpha_{0,m} + \alpha_m^T X) = \\
&= \beta_m \sigma\left(\alpha_{0,m} + \frac{\|\alpha_m\|}{\|\alpha_m\|} \cdot \alpha_m^T X\right) = \beta_m \sigma(\alpha_{0,m} + \|\alpha_m\| \cdot \omega_m^T X),
\end{aligned}$$

where $\omega_m = \frac{\alpha_m}{\|\alpha_m\|}$ is, as before, the m -th unit vector. Equivalent formulations or a shorter notation such as $g_m(V) = \beta_m \sigma(V)$ with a brief explanation of the quantities are also valid solutions.

3b Activation functions

- The activation function is the non-linear transformation applied to the linear model constructed with the inputs. The idea behind it was to mimic what was believed to be the nerve system in a human body. In fact, a nerve was supposed to be "activated" after a certain threshold was reached, and the activation would stop after having received the signal: the same behaviour is reproduced e.g. by a sigmoidal activation function. Note that the activation function does not have to be non-linear, in principle, but a non-linear choice is what makes neural networks powerful on many kinds of different complex problems. Choosing $\sigma = I$, the identity, one ends up with a linear model in the inputs. Common choices of activation function nowadays, on top of the sigmoidal one, are the *arctan* and the *ReLU* (rectified linear) functions.
- The curve for $s = 1/2$ is the blue curve in the figure, while the purple one is obtained for $s = 10$. The parameter s controls the steepness of the activation curve: in fact, a larger s implies a hard activation in form of a step at $v = 0$. This is why this function is also called step or Heaviside function. On the other hand, for small values of s we get an activation curve approximating a linear function. In the limit where s tends to 0, the curve is flattened out and becomes an horizontal line at 0.5; when s tends to ∞ , instead, we get the step function which is 0 for $x \leq 0$ and 1 for $x > 0$. An equally acceptable solution is to define the step function as

Problem 4 Penalized regression

4a Smoothly Clipped Absolute Deviation (SCAD)

- The idea behind the SCAD algorithm is to have a version of lasso where larger coefficients are shrunk less severely. In addition, the solution becomes continuous. Another benefit of the SCAD algorithm is that it is an oracle and therefore $\sqrt{n}$ - consistent estimator.

4b Properties SCAD

- In the above figure, we see a comparison of the size of the theoretical size of a coefficient $\hat{\beta}$ in an ordinary least squares model on the x-axis versus its actual size in the lasso (left) or SCAD model (right). We see that in both cases, $\hat{\beta}$ is set to 0 if it is too small. If it is above a certain threshold,

then $|\hat{\beta}_{\text{lasso}}|$ is always smaller than $|\hat{\beta}_{\text{OLS}}|$ which is the dotted line in the middle of each graph. Similar to lasso, $\hat{\beta}$ needs to be sufficiently large or else it will be set to 0. If it is larger than that, it will be shrunk similarly to lasso. However, at a certain point, the shrinkage is reduced. If $|\hat{\beta}_{\text{SCAD}}|$ is sufficiently large, we recover the OLS estimates since the shrinkage is removed.¹

- The parameter α dictates at which point the coefficients are no longer shrunk as strongly as it would be the case with the typical lasso. This can also be seen in the figure in task 4b. For a certain value for $|y_i|$, the black line is no longer parallel to the dotted line in the center of the graph. Instead, it approaches the dotted line. We can see this in the second equation in 4a) as well. Furthermore, α also dictates, together with λ (see next subtask), when the coefficients are no longer shrunk. Increasing α means that the coefficients $|\beta_i|$ need to be larger in order to not be shrunk. For $\alpha \rightarrow \infty$, SCAD and lasso behave effectively the same way.
- If $|\beta| > \alpha\lambda$, it is no longer shrunk and the OLS estimate is recovered.

4c Oracle properties

There are two properties which an estimator should fulfil (asymptotically) in order to be an oracle estimator. On the one hand, the estimator should identify the right subset model. By that we mean that the true relevant variables are selected and that no meaningless variables are included.

On the other hand, the fitting procedure should converge to the right value for the coefficients. As an example, we have seen in the previous sub-task that lasso does not fulfill this condition. The absolute value of the coefficients is always less than its true value, except for the null case. In contrast, the coefficients of the SCAD procedure do converge asymptotically.

Stating the oracle properties in a mathematically correct manner (using words) is also a valid answer.

¹This is also the reason why SCAD is a $\sqrt{n}$ -consistent estimator (though this is not part of the expected answer).

SKETCH of the SOLUTIONS

STK-IN4300/STK-IN9300 - 2021

Problem 1

- a Elastic Net is a penalised regression approach, in which the penalty term is a mixture of L_1 and L_2 penalties. For this reason, it can be seen as a compromise between lasso and ridge regression. The amount of L_1 penalty is controlled by the hyperparameter λ_1 , so that Elastic Net is equivalent to lasso if $\lambda_1 = 0$. Similarly, the amount of L_2 penalty is also controlled by an hyperparameter, in this case λ_2 . When λ_2 is equal to 0, Elastic Net is equivalent to ridge regression.

As a combination between L_1 and L_2 penalties, Elastic Net has the advantage to have both the variable selection feature of lasso and the ability of handling correlation of ridge regression. The main disadvantage is related to the presence of two hyperparameters: it is computationally costly to tune two hyperparameters and it is not so easy to find the best combination of the two.

- b Like Elastic Net, Bridge regression is a penalize regression approach that relies on two hyperparameters. When $1 \leq q \leq 2$, in particular, it can also be seen as a compromise between the lasso ($q = 1$) and the ridge regression($q = 2$).

The difference, in the case of $1 \leq q \leq 2$, is related to the form of the constraints: while similar in shape, that of elastic net has non-differentiable points that lead to sparser models, as some regression coefficients are forced to be exactly 0. When $1 \leq q \leq 2$, this is not the case for bridge regression. The two approaches are equivalent when $\lambda_1 = 0$, $q = 1$ and $\lambda_2 = \lambda$ and when $\lambda_2 = 0$, $q = 2$ and $\lambda_2 = \lambda$. These case are those corresponding to lasso and ridge regression, respectively.

Problem 2

- a The difference lies in the selection of the variables. Ho's approach uses the same m variables for the entire construction of the tree on each bootstrap sample, Breiman's version samples the m variables at each tree split.

We expect that Ho's algorithm performs worse than Breiman's one in this specific simulation example: since the selection of the m variables happens only once per tree, there are more chances that all the splits are done only considering noisy variables.

- b The random forest regression estimator is defined as the expected value of the single tree, such that

$$\hat{f}_{RF}(x) = E_{\Theta|Z}[T(x; \Theta(Z))],$$

where Θ includes all the characteristics of the individual sample tree (split variables, cut-points, leaves' values).

Then the bias is

$$f(x) - E_Z[E_{\Theta|Z}[T(x; \Theta(Z))]] = f(x) - E_{\Theta,Z}[T(x; \Theta(Z))]$$

i.e., the bias of the individual sample tree.

About the unpruned tree grown to Z , it has smaller bias because it allows more complex estimated functions: the individual trees that compose the random forest are allowed to only use $m < p$ variables at each split. Indeed, in general, increasing m decreases the bias.

Problem 3

- a The effect of the solar radiation on the ozone increases almost linearly until around 230 Langleys, after which it seems to have a negative effect, i.e. a larger amount of radiations exceeding 230 Langleys will mean a smaller concentration of ozone. If the model had only considered a linear effect, this change would have not been modelled, and a slightly increase in the concentration of ozone would have been modelled for each increase of 1 Langsley in solar radiation.
- b See [solution to exercise 5.7](#).

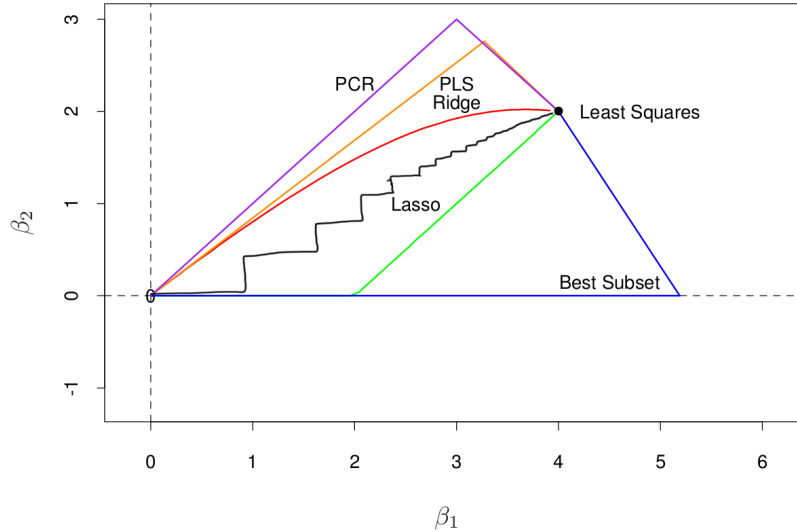
Problem 4

- a The figure describe the learning path of several algorithms in a low-dimensional situation. In particular:

- PCR and ridge regression have paths not so far from each other because they both work on the directions of the principal components. While the former only allows the principal components to be fully in or fully out of the model (in particular the point (3, 3) is reached when the model only includes the first principal component), the latter shrinks the estimates smoothly in both directions, penalising more in the direction of the second principal component. In contrast to PCR, all points of the ridge regression paths are possible, depending on the amount of penalty applied;
- the Lasso path is also a continuous path, where each point corresponds to a value of the penalty parameter, ranging from 0 (the point denoted by “Least Squares” in the figure) to ∞ (the origin). In (2,0) the path changes direction because the penalty term becomes weak enough to allow both variables to enter into the model;
- the Best subset estimate of $\hat{\beta}_1$ is larger when $\hat{\beta}_2 = 0$ than the corresponding value in the Least Squares point due to the positive correlation between X_1 and X_2 : while X_2 is not included, the model tries to capture the common effect of X_1 and X_2 by increasing the value of $\hat{\beta}_1$.

b The path is stepwise, with decreasing step size, like:

$$\rho = 0.5$$



The component-wise boosting approach with a linear base learner updates at each iteration only one component. Therefore the path has a “step” behaviour. Based on the value of the hyperparameter “boosting step size”, these steps can be larger or smaller, but their size decreases

while approaching the Least Squares point, which is only reached in an infinite number of steps.

Problem 5

- a The bias-variance trade-off concept describes the situation for which the minimization of the prediction error must take into account to two terms, bias and variance, that act in opposite way: increasing the model complexity the bias reduces but the variance increases, and the other way around when the model complexity decreases. Finding the best model reduces to find the best point in which the two terms balance.

In gradient boosting, this is mainly associated to the number of boosting steps m_{stop} : given a boosting step size ν , increasing the number of boosting steps leads to a smaller bias but a larger variance, and the other way around if m_{stop} is reduced. The parameter ν , instead, mainly control the speed of this process: larger values of ν mean a faster bias decrease (and variance increase) for each boosting step, while lower values has the opposite effect (slower bias decrease/variable increase for each boosting step).

- b Since we want to use the negative log-likelihood from a Poisson distribution and $\lambda(x) = \exp\{f(x)\}$,

$$\begin{aligned} L(y, f(x)) &= -\log \left(\frac{\exp\{-\exp\{f(x)\}\} \exp\{f(x)\}^y}{y!} \right) \\ &= \exp\{f(x)\} - y \log(\exp\{f(x)\}) + \log(y!) \\ &= e^{f(x)} - y \cdot f(x) + \log(y!). \end{aligned}$$

The negative gradient is then

$$u(y, f(x)) = -\frac{\partial L(y, f(x))}{\partial f(x)} = -e^{f(x)} + y.$$

Problem 6

- a The suggestion of using a large number of hidden units goes together with a proper regularization approach: if the number of hidden units is too small, the Neural Network is not able to fully capture the function complexity, even without regularization. It is therefore suggested to have a large number of hidden units, that can potentially capture the complexity, and have a regularization approach to “put down” the unnecessary units. In the figure, the weight decay parameter is too

small, for this reason the Neural Networks with larger amount of hidden units overfit. the solution is to increase the value of the weight decay parameter.

- b Smaller the weights, closer the Neural Network to the linear model. Increasing values of weights allow the Neural Network to capture non-linearities, increasing the model complexity.

SKETCH of the SOLUTIONS

STK-IN4300/STK-IN9300 - 2019

Problem 1

- a Ridge regression, because it performs better in the case of many variables with small effect (which can be seen from $\beta \sim N(0, 2)$) and if there is a strong correlation among the variables ($\rho = 0.9$).
- b Without knowing that there are many variables with small effect (i.e., we miss the information on the β s), it is safer to use lasso “betting on sparsity”: if there are only a few variables with a large effect, it may strongly outperform ridge; if the situation is similar to the one at point (a), it will not perform much worse than ridge regression.

NOTE: if there was good reasoning behind the choice of ridge regression, the answer was considered correct. Example of good reasoning: “**Despite** the fact that I do not know the effect of the variables, which would let me choose lasso, I prefer to use ridge because I want my model to handle the correlation in a better way, and, anyway, part of the effect of a few potential strong variables is shared with all the variables due to correlation”.

- c Yes, it would allow having a sparse model due to the L_1 penalty, with a better handling of the correlation among variables thanks to the L_2 penalty.

Problem 2

- a The idea is to start from a parametric estimate of the density, $f_0(x, \theta)$, and later correct it with a non-parametric part $(\sum_{i=1}^N \frac{K_\lambda(x_i - x)}{f_0(x_i, \hat{\theta})})$. So one could have a first good global approximation, and let the non-parametric part focus on the discrepancies between the parametric part and the true density function.
- b In this case, one may want to use an exponential distribution, that solves the boundary problem: being equal to 0 for impossible values (there cannot be a negative quantity of wine drunk by a person in one year), it prevents the non-parametric part to give positive density to areas outside the support.

Exercise 3

- a The procedure is wrong because it uses the test data in the model construction procedure, and the error computed in the second point is underestimated. The cross-validation procedure to find the best value of the tuning parameter, indeed, must use only the training data. The correct procedure is to use a nested cross-validation procedure: for example, use LOOCV and, in each iteration, perform the K-fold cross-validation on the $n - 1$ observations currently used as a training set to find the best tuning parameter.
- b The larger the number of folds, smaller the bias, because the training sets will be larger, more similar to the whole training set; and larger the variance, because the training sets will be very similar to each other, so the estimate will be very sample-specific: with a different dataset, one could obtain a very different estimate of the error.

NOTE: in this point was very important to show to have understood that the variance refers to the whole cross-validation estimator and not to the fold-specific estimator.

Exercise 4

- a We expect the orange line to go under the green line, i.e., the result of consensus being worse than the individual guess. It shows that when the estimators computed in each repetition of bagging are worse than guessing at random, in average bagging actually performs worse than the single estimator, as it “makes stronger” a bad solution.
- b Since 35 out of 50 members vote randomly (so they are correct with probability $1/4$) and the remaining 15 with probability p ,

$$EC = 10 \times \left(\frac{35}{50} \cdot \frac{1}{4} + \frac{15}{50} \cdot p \right)$$

where EC is the expected number of correct answers in the 10 categories. With $p = 0$, $EC = 10 \times 7/40 = 7/4$.

- c Since $Var \left[\frac{1}{B} \sum_{b=1}^B X_b \right] = E \left[\left(\frac{1}{B} \sum_{b=1}^B X_b \right)^2 \right] - \left(E \left[\frac{1}{B} \sum_{b=1}^B X_b \right] \right)^2$, with the last term simply equal to μ^2 , we should focus on the first term on

the right hand side,

$$\begin{aligned}
E \left[\left(\frac{1}{B} \sum_{b=1}^B X_b \right)^2 \right] &= \frac{1}{B^2} E \left[\sum_{b=1}^B X_b^2 + 2 \sum_{b \neq c} X_b X_c \right] \\
&= \frac{1}{B^2} E \left(\sum_{b=1}^B E[X_b^2] + 2 \sum_{b > c} E[X_b X_c] \right) \\
&= \frac{1}{B^2} B(\mu^2 + \sigma^2) + \frac{2}{B^2} \binom{B}{2} E[X_b X_c] \\
&= \frac{1}{B}(\mu^2 + \sigma^2) + \frac{2}{B^2} \frac{B(B-1)}{2}(\rho\sigma^2 + \mu^2)
\end{aligned}$$

because, given that $E[X_b] = E[X_c] = \mu$,

$$\begin{aligned}
\rho &= \frac{E[(X_b - \mu)(X_c - \mu)]}{\sigma^2} \\
&= \frac{E[X_b X_c] - \mu E[X_c] - \mu E[X_b] + \mu^2}{\sigma^2} \\
&= \frac{E[X_b X_c] - \mu^2}{\sigma^2}
\end{aligned}$$

so that $E[X_b X_c] = \rho\sigma^2 + \mu^2$.

Substituting in the first expression,

$$\begin{aligned}
Var \left[\frac{1}{B} \sum_{b=1}^B X_b \right] &= E \left[\left(\frac{1}{B} \sum_{b=1}^B X_b \right)^2 \right] - \left(E \left[\frac{1}{B} \sum_{b=1}^B X_b \right] \right)^2 \\
&= \frac{1}{B}(\mu^2 + \sigma^2) + \frac{2}{B^2} \frac{B(B-1)}{2}(\rho\sigma^2 + \mu^2) - \mu^2 \\
&= \frac{1}{B}(\mu^2 + \sigma^2) + \frac{B-1}{B}(\rho\sigma^2 + \mu^2) - \mu^2 \\
&= \frac{\mu^2 + \sigma^2 + B\rho\sigma^2 - \rho\sigma^2 + B\mu^2 - \mu^2 - B\mu^2}{B} \\
&= \frac{\sigma^2 + B\rho\sigma^2 - \rho\sigma^2}{B} \\
&= \frac{\sigma^2}{B} + \rho\sigma^2 - \frac{\rho\sigma^2}{B} \\
&= \rho\sigma^2 + \frac{(1-\rho)}{B}\sigma^2
\end{aligned}$$

Exercise 4

- a If the boosting step size is too small, it would take too many iterations to reach the best estimate in terms of prediction error. We see that in the first plot, in which we do not find the minimum within the

considered number of boosting iterations. On the other hand, if the boosting step size is too large, we do not have enough shrinkage and we risk to overfit already at the beginning, as we can see in the fourth figure. When the boosting step size has the right magnitude, we see the typical behaviour of the prediction error due to the combination of bias and variance. By increasing the number of boosting steps, initially we strongly decrease the bias, and the prediction error decreases despite the increase of the variance. At a certain point, however, the bias reduction gets small, and the increase of the variance will dominate. As a consequence, the prediction error increases (we are overfitting). In this case, one may want to choose $\nu = 0.1$ and 8 iterations: one could get a similar result with $\nu = 0.01$ and 55 boosting iterations, but it would need more time without any improvement in terms of performance.

NOTE: those who chose $\nu = 0.01$ and number iteration = 55 providing a good reason (e.g., the curve is more smooth therefore any small error in the choice of the number of steps will affect way less the algorithm) got full points.

- b In the componentwise version of boosting one dimension is updated at each iteration. Advantages includes the possibility to implement boosting when we have more variables than observations (high-dimensional data), automatic variable selection (the irrelevant dimensions are never selected as the best direction to get improved and therefore never included in the model), and the possibility to use different base-learners for each dimension (e.g., linear effects and splines).

SKETCH of the SOLUTIONS

STK-IN4300/STK-IN9300 - 2019

Problem 1

- a Both ridge regression and principal component regression act on the principal components of the X matrix: the former by shrinking their regression coefficients, the latter by setting them to 0. In particular, ridge regression shrinks more the components with smaller eigenvalues, while the only “shrinkage effect” of the principal component regression is given by the elimination of the information related to the principal components whose coefficients are set to 0.
- b
- Because the intercept is excluded from the penalization, as it makes no sense to shrink it toward 0;
 - one obtains the group lasso and the ordinary lasso, respectively;
 - “bet on sparsity” is that principle for which it is preferable to use a procedure which assumes a sparse truth over one that does not, because the former performs better if the problem is actually sparse, while both procedures tend to perform badly in a dense problem.
- c Bridge regression is a penalize regression approach which uses a penalty of the form $\sum_{i=1}^p |\beta_i|^q$, where p is the number of explanatory variables and β_i the regression coefficients. When $1 \leq q \leq 2$, the penalty can be seen as a compromise between the lasso ($q = 1$) and the ridge ($q = 2$) penalties.

Similarly, elastic net is also a compromise between lasso and ridge regression: in this case, a mixture of L_1 and L_2 penalties is used, with a hyperparameter controlling the ratio between the two penalties. The resulting methods is supposed to enclose the advantages of both penalties (mainly, variable selection and a better handling of correlation, respectively).

The difference in the models that one obtains by applying bridge regression and elastic net is related to the form of the constraints: while similar, that of elastic net has non-differentiable corners that lead to sparser models, as some regression coefficients are forced to be exactly 0.

Problem 2

- a Consider three binary classification trees for a new point x_0 . Two of them classifies it as 0 with probability 0.55 (and 1 with probability 0.45), the third with probability 0.10 (so 1 with probability 0.90). When aggregating based on the estimated class, x_0 will be classified as 0 (two votes versus one). In contrast, by considering the probabilities, 1 (averaging the probability, one obtains 0.40 for 0, 0.60 for 1).
- b Since random forests only consider a subset of variables for the tree splitting, an increasing number of noisy variables increases the probability of having subsets which do not include any variable correlated with the outcome.
- c The left-hand side of the equation is the signal-to-noise ratio (larger the value, larger the ratio), while the right-hand side is a quantity related to the shrinkage of the base learner: the stronger the shrinkage (i.e., the smaller λ_k), the smaller the quantity. Therefore, in order for boosting to bring improvement, the signal must be large with respect to the noise, or the shrinkage sufficiently strong.
- d
 - The main tuning parameters are the number of boosting steps m_{stop} and the boosting step size ν ;
 - usually one fixes the boosting step size (e.g., $\nu = 0.1$) and compute the best value of m_{stop} via cross-validation;
 - As long as the boosting step size has the right magnitude, it does not really influence the prediction performance. If it is too small, the procedure results to be too slow, while there is the risk to overfit if its value is set too large. In contrast, the number of boosting steps is extremely important to obtain a good prediction: too many boosting steps lead to overfitting (large variance), not enough boosting steps lead to underfitting (large bias).
 - if the goal is to obtain the best prediction model, a smaller value of the boosting step size requires a larger number of boosting steps, and vice versa.

Exercise 3

- a (see formula (7.9) in the book)
- b (see the proof of Proposition 3 of Bühlmann & Yu, JASA 2003)
- c Model complexity is strictly connected with the concept of bias-variance trade-off: a more complex model tends to have smaller bias and larger variance, while it is the other way around for less complex models. One can see this in kNN. The most complicated model is obtained for $k = 1$, in which there is one average (the predicted outcome) for each observation: in this case, the bias in the training set is 0, but the variance (and, consequently, the expected predicted error) is very large. At the opposite, $k = n$, with n the number of observations, only one average is computed (so the model is the less complex that can be fitted), causing a very large bias and a large expected prediction error (despite the small variance). To obtain a good bias-variance trade-off, the right model complexity must be found.

SKETCH of the SOLUTIONS

STK-IN4300/STK-IN9300 - 2018

Exercise 1

a When using the squared-error loss, the expected prediction error can be decomposed into three parts:

- an irreducible error, which cannot be avoided;
- the squared bias, where bias is the difference between the average of the estimate and the true mean;
- a variance term, the expected square deviation of our estimate from its mean.

When minimizing the expected prediction error, we are in a situation in which if we reduce the variance component, for example by adding constraints to the model space, we increase the squared bias, and vice versa. This is called bias-variance trade-off. In the picture this is represented by lines and circles: when adding constraints, o.e. when we move from the model space to the restricted model space, we increase the bias (represented by the line named “Estimation Bias”) but we reduce the variance (the radius of the circle around the “Closest fit in population” point is smaller than that around the corresponding point on the restricted model space).

b Bias:

$$E[\hat{\beta}_{LS}] = \beta$$

$$\begin{aligned} E[\hat{\beta}_{\text{ridge}}(\lambda)] &= E[(X^T X + \lambda I_p)^{-1} X^T y] \\ &= E[(I_p + \lambda(X^T X)^{-1})^{-1} \underbrace{(X^T X)^{-1} X^T y}_{\hat{\beta}_{LS}}] \\ &= \underbrace{(I_p + \lambda(X^T X)^{-1})^{-1}}_{w_\lambda} E[\hat{\beta}_{LS}] \\ &= w_\lambda \beta \implies E[\hat{\beta}_{\text{ridge}}(\lambda)] \neq \beta \text{ for } \lambda > 0. \end{aligned}$$

Variance:

$$\text{Var}[\hat{\beta}_{LS}] = \sigma^2 (X^T X)^{-1}$$

$$\begin{aligned} \text{Var}[\hat{\beta}_{\text{ridge}}(\lambda)] &= \text{Var}[w_\lambda \hat{\beta}_{LS}] \\ &= w_\lambda \text{Var}[\hat{\beta}_{LS}] w_\lambda^T \\ &= \sigma^2 w_\lambda (X^T X)^{-1} w_\lambda^T. \end{aligned}$$

Then,

$$\begin{aligned}
\text{Var}[\hat{\beta}_{\text{LS}}] - \text{Var}[\hat{\beta}_{\text{ridge}}(\lambda)] &= \sigma^2 [(X^T X)^{-1} - w_\lambda (X^T X)^{-1} w_\lambda^T] \\
&= \sigma^2 w_\lambda [(I_p + \lambda(X^T X)^{-1})(X^T X)^{-1}(I_p + \lambda(X^T X)^{-1})^T - (X^T X)^{-1}] w_\lambda^T \\
&= \sigma^2 w_\lambda [(X^T X)^{-1} + 2\lambda(X^T X)^{-2} + \lambda^2(X^T X)^{-3} - (X^T X)^{-1}] w_\lambda^T \\
&= \sigma^2 w_\lambda [2\lambda(X^T X)^{-2} + \lambda^2(X^T X)^{-3}] w_\lambda^T > 0
\end{aligned}$$

(since all terms are quadratic and therefore positive)

$$\implies \text{Var}[\hat{\beta}_{\text{ridge}}(\lambda)] \leq \text{Var}[\hat{\beta}_{\text{LS}}]$$

Exercise 2

- a The procedure is incorrect because the prediction error is computed on observations already used to train the prediction rule. This lead to underestimation of the error (writing “too optimistic” was acceptable).

A possible solution is to compute the prediction error only on those observations (in average 36.8% of the original sample) not included in the bootstrap sample. Since this approach leads to overestimating the prediction error, solutions like those described in the point (b) has been implemented.

- b The **0.632 bootstrap** procedure addresses the problem of overestimation of the correct procedure described in point (a) by averaging it (with weight 0.632 and 0.368, respectively) with the training error (underestimated error). The result is a sort of compromise between overestimation and underestimation.

In formula:

$$\widehat{\text{Err}}^{(0.632)} = 0.632 \widehat{\text{Err}}^{(1)} + 0.368 \bar{\text{err}},$$

where $\bar{\text{err}}$ is the training error and $\widehat{\text{Err}}^{(1)}$ the corrected procedure described at point (a).

Since the 0.632 and 0.382 weights may not be the best choice (e.g., in case of complete overfitting in the training set), the **0.632+ bootstrap** has been developed. In the latter procedure,

$$\widehat{\text{Err}}^{(0.632+)} = \hat{w} \widehat{\text{Err}}^{(1)} + (1 - \hat{w}) \bar{\text{err}},$$

the weights depend on the relative overfitting rate, so the **0.632+ bootstrap** can be seen as a better compromise between the overestimation and underestimation of the prediction error done by the corrected procedure described at point (a) and the training error, respectively.

Exercise 3

- a The penalization term penalizes curves too “wiggly”, reducing the model complexity by penalizing curves with high curvature. The amount of penalty is controlled by the tuning parameter λ :

- when $\lambda = 0$ there is no penalization, and it leads to a curve which passes through all the points;
- when $\lambda = \infty$ no curvature is allowed, and it leads to a straight line.

The choice of λ is also a case of bias-variance trade-off: smaller λ , smaller the bias (and higher the variance); larger λ , larger the bias (and smaller the variance).

- b Rewriting equation (1) in terms of θ , i.e. by plugging in $f(x) = \sum_{i=1} N_j(x)\theta_j$, one obtains the form

$$RSS(\theta, \lambda) = (y - N\theta)^T(y - N\theta) + \lambda\theta^T\Omega_N\theta,$$

where $\{N\}_{ij} = N_j(x_i)$ and $\{\Omega_N\}_{jk} = \int N''(t)N''(t)dt$.

Either deriving (and setting the first derivative equal to 0) or recognizing the solution of a regularized ridge regression, one obtains

$$\theta = (N^TN + \lambda\Omega_N)^{-1}N^Ty.$$

Therefore

$$\hat{f} = N(N^TN + \lambda\Omega_N)^{-1}N^Ty,$$

which is linear in y . Knowing that the degrees of freedom of a linear smoother correspond to the trace of the smoothing matrix,

$$\text{dof}(\hat{f}) = \text{trace}(N(N^TN + \lambda\Omega_N)^{-1}N^T).$$

Exercise 4

- a Bagging (Bootstrap AGGREGatING) is a procedure which consists in aggregating (by averaging, by voting, etc.) the results of a prediction rule applied to a number of bootstrap samples generated from the original data. The prediction rule is typically (but not necessarily) a tree.

An advantage of bagging with respect to a single tree is its stability (reduced variance), while in contrast to a boosted tree model bagging is not able to take advantage of the results of the previous iterations to improve the later predictions (e.g., in the context of classification, AdaBoost is able to focus on misclassified observations by weighting them more in the later iterations).

- b Consider a simple case of binary classification in which we aggregate the results of three trees: two trees say that an observation x_i is of class A with probability 0.55, of class B with probability 0.45; the third tree, instead, says A with probability 0.1, B with probability 0.9.

When aggregating by “majority of votes”, x_i is classified as A (two votes against one).

When aggregating by “class probabilities”, x_i is classified as B (average probabilities being 0.4 for A, 0.6 for B).

Exercise 5

- a The solution was seen in class during the 11th lecture (solution of exercise 10.2 of the text book), see also [here](#).
- b The algorithm is called “gradient boosting” and the omitted expression is

$$u_m = - \left. \frac{\partial L(y, f(x))}{\partial f(x)} \right|_{f(x)=f_{m-1}(x)}$$